Министерство образования и науки РФ
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра Информационные технологии и автоматизированные системы

Базы данных

Лабораторная работа № 4

Тема: «Реализация команд DDL, DML и соединений на веб-странице»

Выполнил: студент группы РИС-25-2б
Фокин Алексей
Проверил: доцент кафедры ИТАС
Петренко А.А.

г. Пермь – 2026

СОДЕРЖАНИЕ

ЦЕЛЬ РАБОТЫ **3**

ЗАДАЧИ РАБОТЫ **4**

ЭТАПЫ ВЫПОЛНЕНИЯ **5**

1.1 Просмотр данных (SHOW и SHOW ALL) 8

1.2 Команды DDL: CREATE, ALTER, DROP, TRUNCATE 10

1.3 Команды DML: INSERT, UPDATE, DELETE 17

1.4 Выборка SELECT и соединение INNER JOIN 23

ЗАКЛЮЧЕНИЕ **32**

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ **33**

ПРИЛОЖЕНИЕ А **34**

ЦЕЛЬ РАБОТЫ

Реализовать выполнение основных команд SQL: DDL, DML, а также оператора соединения INNER JOIN через интерфейс веб-страницы.

ЗАДАЧИ РАБОТЫ

1) Изучить команды DDL для создания, модификации и удаления таблиц;
2) Изучить команды DML для добавления, изменения и удаления данных;
3) Разработать страницу, демонстрирующую работу всех команд DDL, DML и оператора INNER JOIN;
4) Вывести результаты работы команд и выборок на страницу браузера;

ЭТАПЫ ВЫПОЛНЕНИЯ

Для выполнения данной лабораторной работы создается отдельная база данных db4. Это необходимо для того, чтобы изолировать потенциально опасные мутирующие операции (такие как DROP TABLE, TRUNCATE, массовое удаление строк и изменение структуры колонок) от рабочей базы предыдущих лабораторных работ. В качестве исходных данных в db4 переносится таблица Table_Fokin с 20 записями, использовавшаяся ранее. Соединение с базой данных устанавливается через драйвер PDO способом, описанным в лабораторной работе № 3.

Общий цикл работы приложения построен следующим образом. Сначала скрипт подключается к СУБД. Затем анализируется массив $_POST: определяется, какая именно кнопка была нажата и какой параметр action был передан формой. В зависимости от значения action программа выбирает одну из ветвей обработки и динамически собирает соответствующий SQL-запрос, экранируя пользовательские данные и идентификаторы. Готовый запрос передается на исполнение серверу PostgreSQL. На финальном шаге результат выборки либо сообщение о статусе операции передается в HTML-разметку и отрисовывается в браузере.

Поскольку значительная часть работы связана с формированием SQL-запросов из пользовательского ввода, в скрипте выделен набор вспомогательных функций (листинг 1). Функция h() экранирует произвольное значение для безопасной вставки в HTML-разметку. Функция post() возвращает значение параметра из массива $_POST либо значение по умолчанию, если параметр не был передан. Функция quoteIdent() оборачивает идентификатор PostgreSQL (имя таблицы или колонки) в двойные кавычки и удваивает внутренние кавычки, что предотвращает SQL-инъекции на уровне структуры запроса. Функции getTables() и getColumns() читают метаданные из системной схемы information_schema и возвращают списки существующих таблиц и их колонок — на их основе строятся все выпадающие списки на странице.

```
function h($value) {
    return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}

function post($name, $default = '') {
    return $_POST[$name] ?? $default;
```

```
}

function quoteIdent($name) {
    return '"' . str_replace('"', '""', $name) . '"';
}

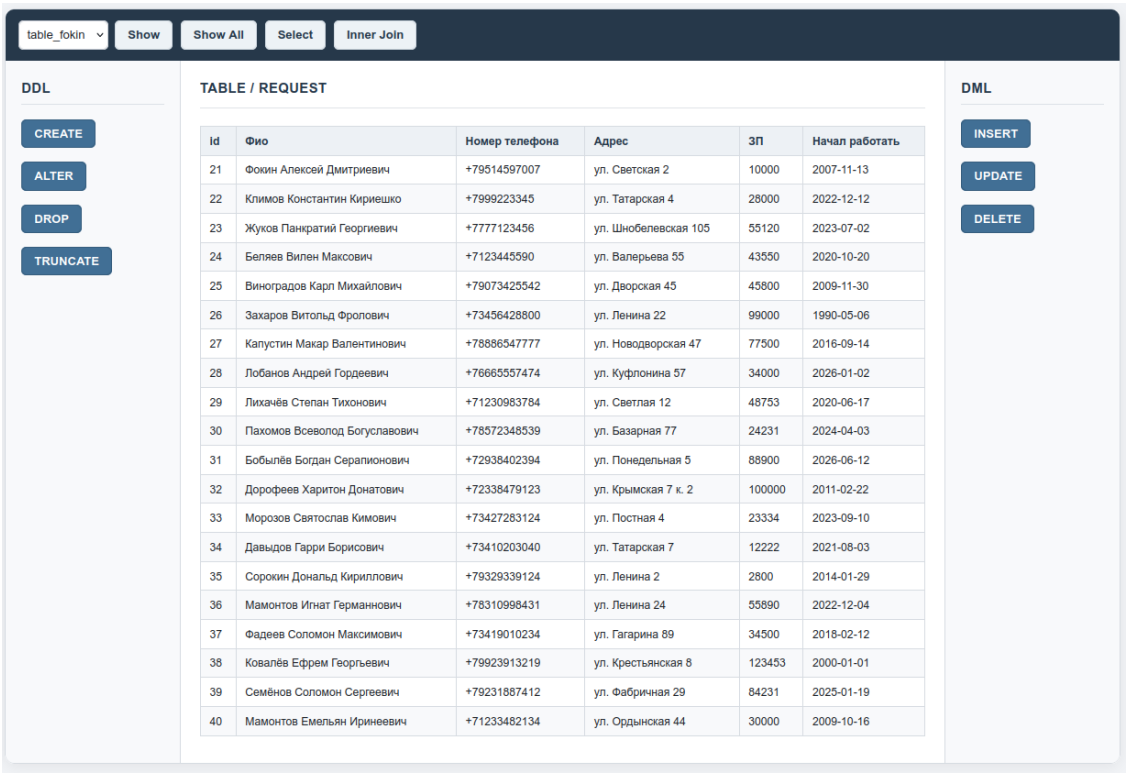
function getTables($pdo) {
    $sql = "SELECT table_name FROM information_schema.tables
            WHERE table_schema = 'public' AND table_type = 'BASE
   TABLE'
            ORDER BY table_name";
    return $pdo->query($sql)->fetchAll(PDO::FETCH_COLUMN);
}

function getColumns($pdo, $table) {
    $sql = "SELECT column_name, data_type, column_default
            FROM information_schema.columns
            WHERE table_schema = 'public' AND table_name = :table
            ORDER BY ordinal_position";
    $stmt = $pdo->prepare($sql);
    $stmt->execute(['table' => $table]);
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

Листинг 1 — Вспомогательные функции для экранирования и чтения метаданных

Помимо этого, определены небольшие генераторы разметки: tableSelect() строит выпадающий список таблиц, columnSelect() — список колонок выбранной таблицы, operatorSelect() — список операторов сравнения (=, <>, <, >, <=, >=, LIKE). Каждая из них также опирается на getTables() и getColumns(), то есть все селекторы на странице формируются динамически на основе запросов к системной схеме.

Внешний вид главной страницы панели управления приведен на рисунке 1.



Id	Фио	Номер телефона	Адрес	ЗП	Начал работать
21	Фокин Алексей Дмитриевич	+79514597007	ул. Светская 2	10000	2007-11-13
22	Климов Константин Кириешко	+7999223345	ул. Татарская 4	28000	2022-12-12
23	Жуков Панкратий Георгиевич	+7777123456	ул. Шнобелевская 105	55120	2023-07-02
24	Беляев Вилен Максович	+7123445590	ул. Валерьева 55	43550	2020-10-20
25	Виноградов Карл Михайлович	+79073425542	ул. Дворская 45	45800	2009-11-30
26	Захаров Витольд Фролович	+73456428800	ул. Ленина 22	99000	1990-05-06
27	Капустин Макар Валентинович	+78886547777	ул. Новодворская 47	77500	2016-09-14
28	Лобанов Андрей Гордеевич	+76665557474	ул. Куфлонина 57	34000	2026-01-02
29	Лихачёв Степан Тихонович	+71230983784	ул. Светлая 12	48753	2020-06-17
30	Пахомов Всеволод Богуславович	+78572348539	ул. Базарная 77	24231	2024-04-03
31	Бобылёв Богдан Серапионович	+72938402394	ул. Понедельная 5	88900	2026-06-12
32	Дорофеев Харитон Донатович	+72338479123	ул. Крымская 7 к. 2	100000	2011-02-22
33	Морозов Святослав Кимович	+73427283124	ул. Постная 4	23334	2023-09-10
34	Давыдов Гарри Борисович	+73410203040	ул. Татарская 7	12222	2021-08-03
35	Сорокин Дональд Кириллович	+79329339124	ул. Ленина 2	2800	2014-01-29
36	Мамонтов Игнат Германнович	+78310998431	ул. Ленина 24	55890	2022-12-04
37	Фадеев Соломон Максимович	+73419010234	ул. Гагарина 89	34500	2018-02-12
38	Ковалёв Ефрем Георгьевич	+79923913219	ул. Крестьянская 8	123453	2000-01-01
39	Семёнов Соломон Сергеевич	+79231887412	ул. Фабричная 29	84231	2025-01-19
40	Мамонтов Емельян Иринеевич	+71233482134	ул. Ордынская 44	30000	2009-10-16

Рисунок 1 — Главная страница панели управления базой данных

При загрузке страницы пользователю доступны: верхняя панель с выпадающим списком таблиц и кнопками быстрого вывода, левая панель команд DDL и правая панель команд DML. В центре находится рабочая область (workspace), в которой отрисовываются формы ввода и результирующие таблицы.

После выполнения любой операции в верхней части рабочей области появляется цветной баннер с результатом: зеленый — при успехе, красный — при ошибке. Общий вид сообщения об успешном выполнении показан на рисунке 2.

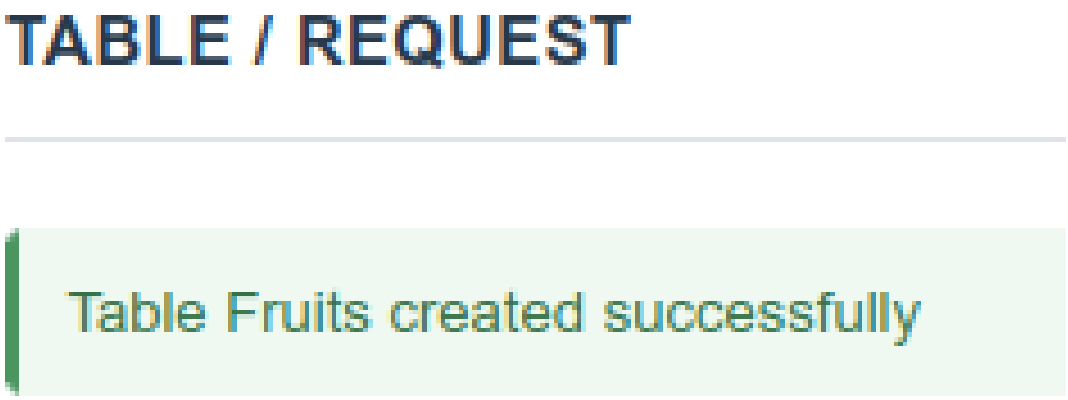


Рисунок 2 — Баннер с сообщением об успешном выполнении операции

1.1 Просмотр данных (SHOW и SHOW ALL)

Просмотр содержимого таблиц реализован двумя кнопками на верхней панели. Кнопка Show выводит все строки выбранной таблицы; кнопка Show All обходит все существующие таблицы в цикле и выводит их одну за другой. Обе операции не изменяют данные и не требуют подтверждения.

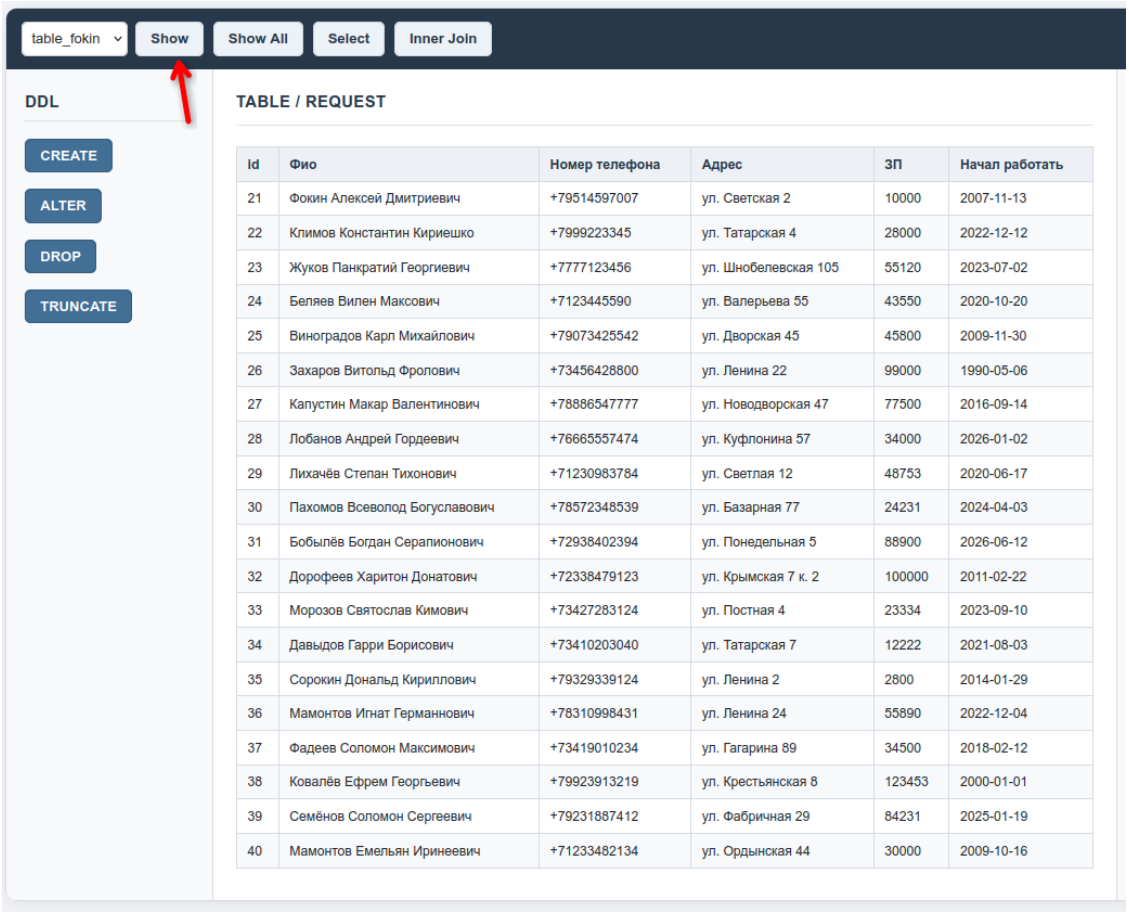
Для вывода содержимого конкретной таблицы пользователь выбирает ее из выпадающего списка и нажимает Show. Форма отправляет action со значением show_execute. Скрипт проверяет наличие таблицы в системной схеме через getTables(), после чего экранирует имя таблицы и запрашивает все поля (листинг 2).

```
if ($action === 'show_execute') {
    $table = post('table');
    if (!in_array($table, getTables($pdo), true)) {
        $error = 'No table in the database.';
    } else {
        $sql = 'SELECT * FROM ' . quoteIdent($table) . ';';
        $rows = $pdo->query($sql)->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

Листинг 2 — Логика просмотра содержимого таблицы

При нажатии Show All аналогичный запрос выполняется в цикле для

каждой найденной таблицы, возвращая сразу все таблицы базы на один экран. Результат одиночного Show для таблицы Table_Fokin приведен на рисунке 3.



id	Фио	Номер телефона	Адрес	ЗП	Начал работать
21	Фокин Алексей Дмитриевич	+79514597007	ул. Светская 2	10000	2007-11-13
22	Климов Константин Кириешко	+7999223345	ул. Татарская 4	28000	2022-12-12
23	Жуков Панкратий Георгиевич	+7777123456	ул. Шнобелевская 105	55120	2023-07-02
24	Беляев Вилен Максович	+7123445590	ул. Валерьева 55	43550	2020-10-20
25	Виноградов Карл Михайлович	+79073425542	ул. Дворская 45	45800	2009-11-30
26	Захаров Витольд Фролович	+73456428800	ул. Ленина 22	99000	1990-05-06
27	Капустин Макар Валентинович	+78886547777	ул. Новодворская 47	77500	2016-09-14
28	Лобанов Андрей Гордеевич	+76665557474	ул. Куфлонина 57	34000	2026-01-02
29	Лихачёв Степан Тихонович	+71230983784	ул. Светлая 12	48753	2020-06-17
30	Пахомов Всеволод Богуславович	+78572348539	ул. Базарная 77	24231	2024-04-03
31	Бобылёв Богдан Серапионович	+72938402394	ул. Понедельная 5	88900	2026-06-12
32	Дорофеев Харитон Донатович	+72338479123	ул. Крымская 7 к. 2	100000	2011-02-22
33	Морозов Святослав Кимович	+73427283124	ул. Постная 4	23334	2023-09-10
34	Давыдов Гарри Борисович	+73410203040	ул. Татарская 7	12222	2021-08-03
35	Сорокин Дональд Кириллович	+79329339124	ул. Ленина 2	2800	2014-01-29
36	Мамонтов Игнат Германнович	+78310998431	ул. Ленина 24	55890	2022-12-04
37	Фадеев Соломон Максимович	+73419010234	ул. Гагарина 89	34500	2018-02-12
38	Ковалёв Ефрем Георгьевич	+79923913219	ул. Крестьянская 8	123453	2000-01-01
39	Семёнов Соломон Сергеевич	+79231887412	ул. Фабричная 29	84231	2025-01-19
40	Мамонтов Емельян Иринеевич	+71233482134	ул. Ордынская 44	30000	2009-10-16

Рисунок 3 — Отображение содержимого таблицы Table_Fokin

Видим, что в рабочей области отрисовалась таблица со всеми двадцатью записями, заполненными в предыдущей лабораторной работе. Далее проверим работу Show All при наличии в базе двух таблиц — Table_Fokin и Fruits (последняя создается в следующем подразделе). Результат показан на рисунке 4.

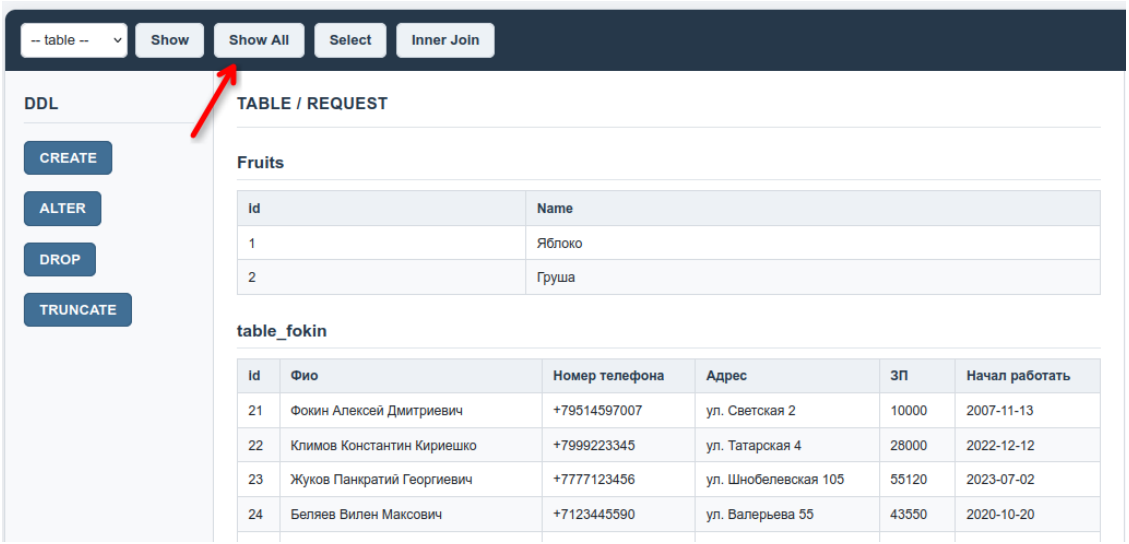


Рисунок 4 — Вывод всех таблиц базы командой Show All

Обе таблицы выводятся последовательно, каждая под своим заголовком. Поскольку список таблиц и их содержимое формируются на сервере без применения JavaScript, вся динамика страницы обеспечивается исключительно запросами к базе данных и повторной отрисовкой HTML.

1.2 Команды DDL: CREATE, ALTER, DROP, TRUNCATE

Группа DDL (Data Definition Language) отвечает за определение и изменение структуры объектов базы данных. В работе рассматриваются четыре команды: CREATE TABLE, ALTER TABLE, DROP TABLE и TRUNCATE TABLE.

Команда CREATE TABLE создает новую таблицу. Общий синтаксис: CREATE TABLE имя_таблицы (колонка1 тип1 [ограничения], колонка2 тип2, …). Создание таблицы в панели разбито на два этапа. Сначала пользователь вводит имя будущей таблицы и количество столбцов, отправляя действие create. Скрипт генерирует форму ввода параметров для каждого столбца: имя, тип данных и чекбокс первичного ключа. Внешний вид первого шага приведен на рисунке 5.

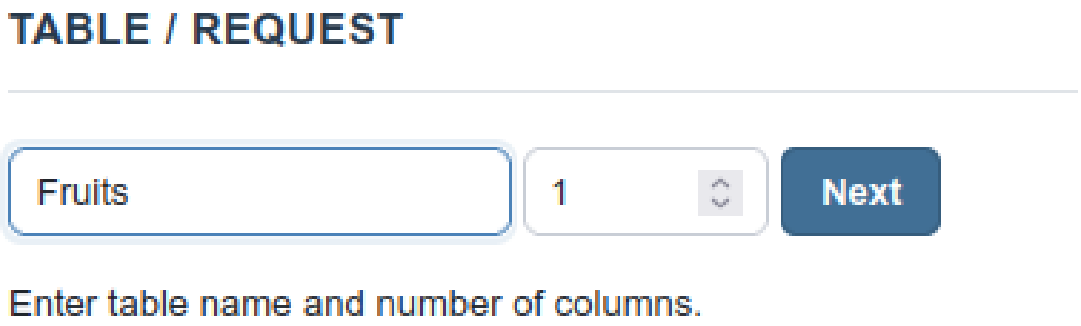


Рисунок 5 — Первый шаг создания таблицы: имя и число столбцов

На втором шаге формируется сетка столбцов (рисунок 6). Для каждого столбца задаются имя, тип из выпадающего списка и признак первичного ключа.

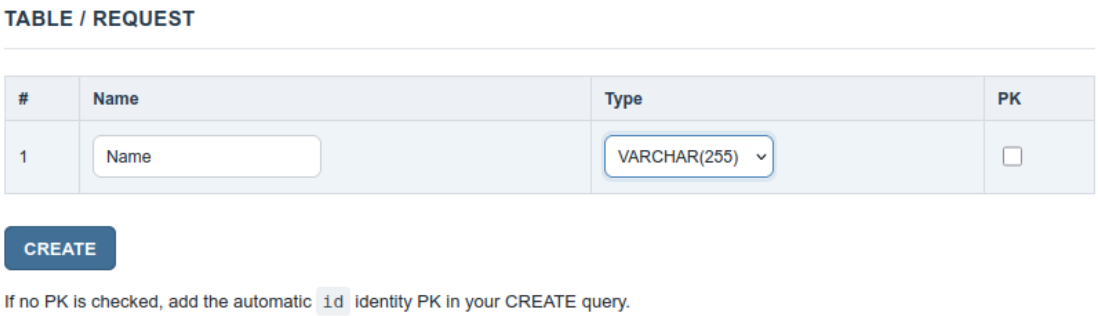


Рисунок 6 — Второй шаг создания таблицы: параметры столбцов

При нажатии CREATE управление переходит в блок create_execute. Скрипт собирает массив колонок; если ни один первичный ключ не был выбран, автоматически добавляется суррогатный ключ id SERIAL PRIMARY KEY. Имена колонок экранируются функцией quoteIdent(), после чего готовая строка CREATE TABLE передается в метод $pdo->exec() (листинг 3):

```
for ($i = 0; $i < $count; $i++) {
    $argName = $names[$i] ?? "";
    $argType = $types[$i] ?? "";
    $arg = quoteIdent($argName) . ' ' . $argType;
```

```
    if (!empty($pks[$i])) {
        $arg .= ' PRIMARY KEY';
    }
    $args[] = $arg;
}
$sql = 'CREATE TABLE ' . quoteIdent($table) . '(' . implode(', ',
    $args) . ');';
$pdo->exec($sql);
```

Листинг 3 — Сборка запроса CREATE TABLE

Создадим таблицу Fruits с одним столбцом Name типа VARCHAR. Первичный ключ не отмечаем, поэтому система автоматически добавит служебный столбец id. После подтверждения таблица создается, и в верхней части рабочей области появляется зеленый баннер с текстом Table Fruits created successfully (рисунок 7).

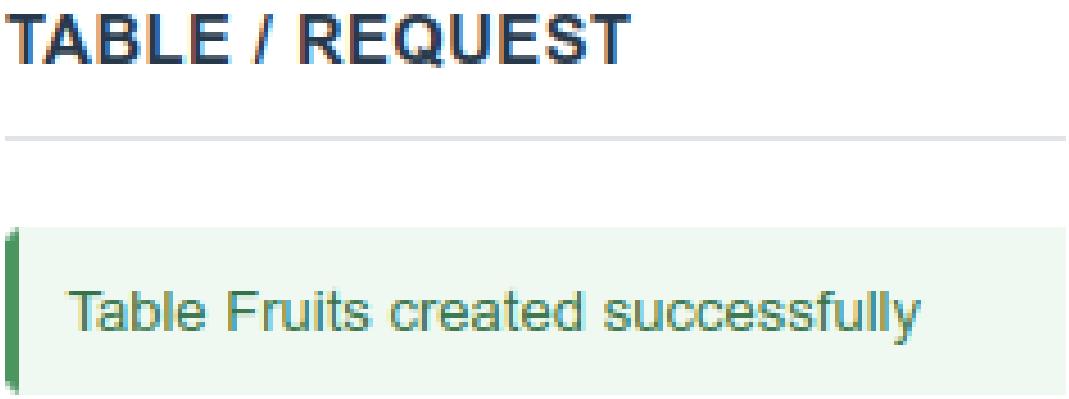


Рисунок 7 — Успешное создание таблицы Fruits

Далее проверим обработку ошибок. Попробуем повторно создать уже существующую таблицу Table_Fokin. Скрипт выполняет проверку in_array($table, getTables($pdo), true) и, обнаружив совпадение, возвращает красный баннер с диагностикой (рисунок 8).

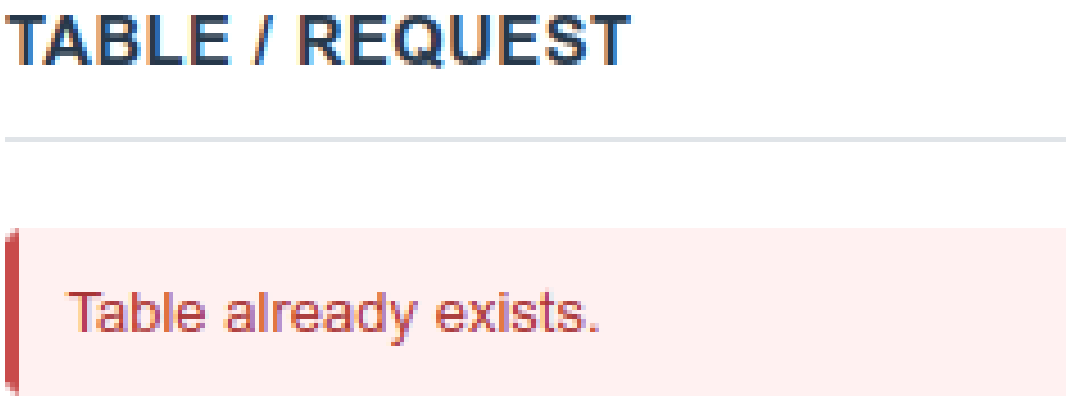


Рисунок 8 — Сообщение об ошибке при попытке повторного создания таблицы

Команда ALTER TABLE модифицирует структуру уже существующей таблицы. Общий синтаксис: ALTER TABLE имя ADD|ALTER|DROP COLUMN имя [тип]. В панели пользователь выбирает таблицу, тип действия, имя столбца и целевой тип данных. Внешний вид формы приведен на рисунке 9.

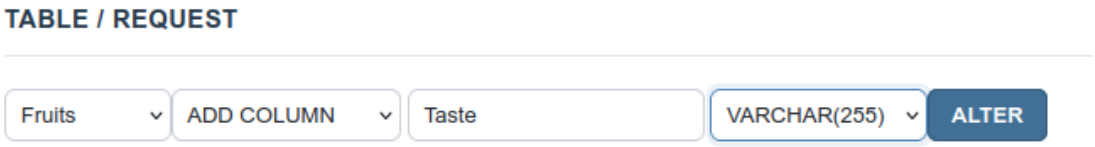


Рисунок 9 — Рабочая область команды ALTER TABLE

Следует заметить, что выпадающий список типов отображается и при выборе действия DROP COLUMN, хотя в этом случае тип не требуется. Это связано с тем, что страница полностью серверная и не использует JavaScript: скрыть или показать поле в зависимости от выбранного действия без перезагрузки страницы невозможно, поэтому поле присутствует всегда и просто игнорируется сервером при соответствующем действии.

На сервере в блоке alter_execute формируется нужная конструкция ALTER TABLE (листинг 4):

```
$sql = 'ALTER TABLE ' . quoteIdent($table) . ' ';
if ($alter_type === 'add') {
    $sql .= 'ADD ' . quoteIdent($newColName) . ' ' . $newColType;
} elseif ($alter_type === 'alter') {
    $sql .= 'ALTER COLUMN ' . quoteIdent($newColName) . ' TYPE '
   . $newColType;
} elseif ($alter_type === 'drop') {
    $sql .= 'DROP COLUMN ' . quoteIdent($newColName);
}
$pdo->exec($sql . ';');
```

Листинг 4 — Сборка запроса ALTER TABLE

Добавим в ранее созданную таблицу Fruits колонку Taste типа VARCHAR. После выполнения запроса структура таблицы обновляется без ошибок, а в рабочей области появляется соответствующее подтверждение (рисунок 10).

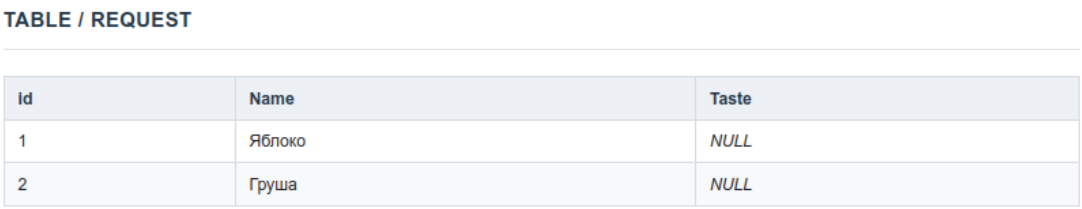


TABLE / REQUEST

Id	Name	Taste
1	Яблоко	*NULL*
2	Груша	*NULL*

Рисунок 10 — Результат выполнения команды ALTER TABLE

Команда TRUNCATE TABLE удаляет все строки таблицы, сохраняя саму структуру и определение колонок. В отличие от DELETE, команда TRUNCATE не пишет построчно в журнал и работает значительно быстрее на больших объемах. Синтаксис: TRUNCATE TABLE имя. Для выполнения операции пользователь выбирает целевую таблицу через форму simpleTableWorkspace() (рисунок 11).

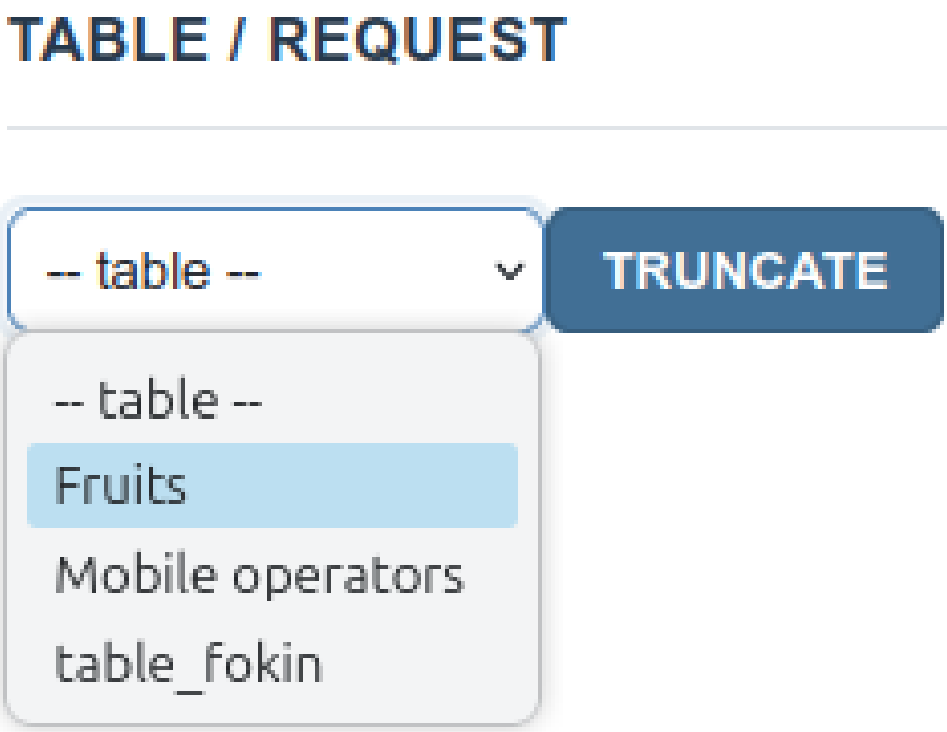


Рисунок 11 — Рабочая область команды TRUNCATE

Серверный обработчик truncate_execute сводится к одной строке (листинг 5):

```php
if ($action === 'truncate_execute') {
    $pdo->exec('TRUNCATE TABLE ' . quoteIdent($table));
    $message = 'Table ' . h($table) . ' truncated successfully.';
}
elseif ($action === 'drop_execute') {
    $pdo->exec('DROP TABLE ' . quoteIdent($table));
    $message = 'Table ' . h($table) . ' dropped successfully.';
}
```

Листинг 5 — Обработка команд TRUNCATE и DROP

Очистим таблицу Fruits командой TRUNCATE. Все строки удаляются, структура остается нетронутой, о чем сообщает зеленый баннер (рисунок 12).

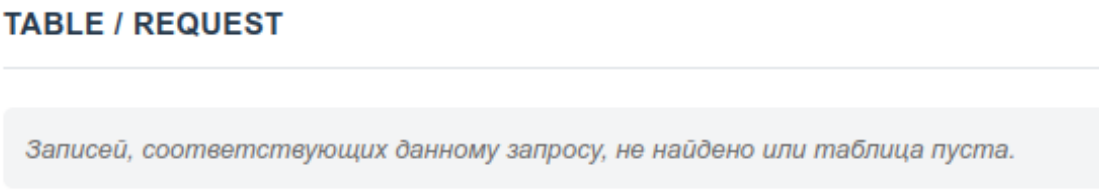


Рисунок 12 — Результат выполнения команды TRUNCATE

Команда DROP TABLE полностью удаляет таблицу и все связанные с ней данные из схемы базы. Синтаксис: DROP TABLE имя. Рабочая область совпадает с областью TRUNCATE и приведена на рисунке 13.

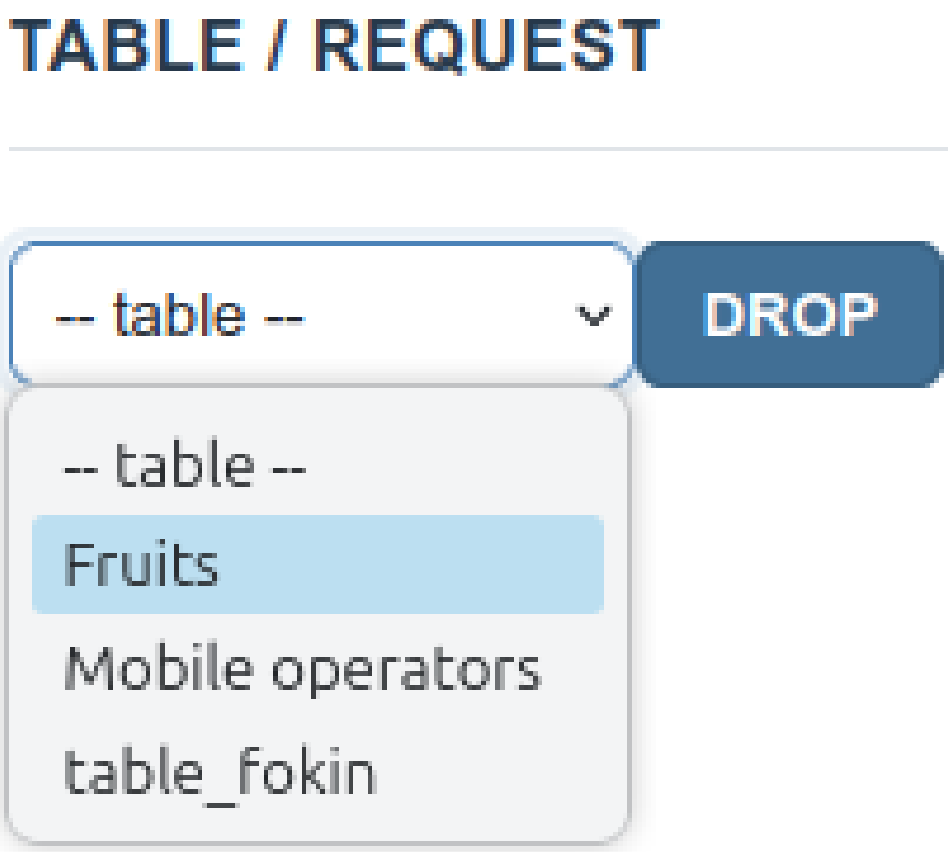


Рисунок 13 — Рабочая область команды DROP

Полностью удалим таблицу Fruits. После подтверждения в списке таблиц она больше не отображается (рисунок 14). Поскольку работа ведется в изолированной базе db4, операции удаления не затрагивают таблицы предыдущих лабораторных работ.

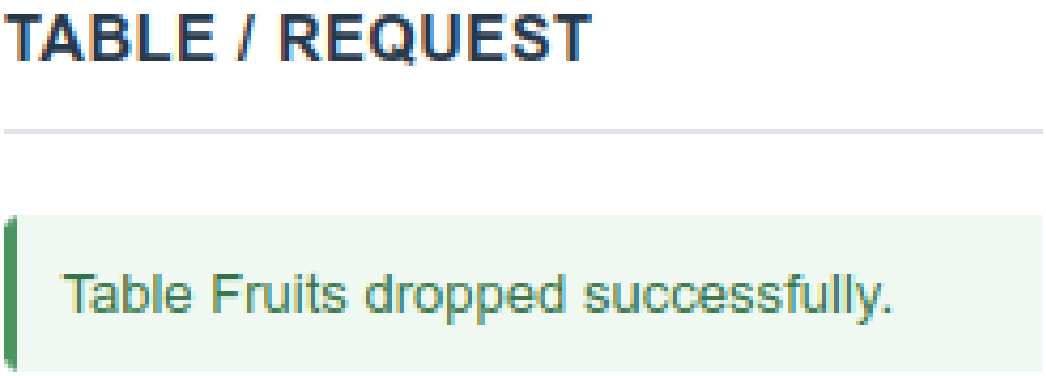


Рисунок 14 — Результат выполнения команды DROP

1.3 Команды DML: INSERT, UPDATE, DELETE

Группа DML (Data Manipulation Language) объединяет команды работы с данными внутри таблиц. В лабораторной работе рассматриваются INSERT (добавление строк), UPDATE (изменение значений), DELETE (удаление строк). Выборка SELECT рассматривается отдельно в следующем подразделе.

Команда INSERT INTO добавляет в таблицу одну или несколько новых строк. Общий синтаксис: INSERT INTO имя (колонка1, колонка2, …) VALUES (значение1, значение2, …), …. В панели вставка выполняется в два шага. Сначала пользователь выбирает таблицу и указывает количество добавляемых записей (рисунок 15).



Рисунок 15 — Первый шаг вставки: выбор таблицы и числа строк

Затем скрипт считывает метаданные таблицы через getColumns() и строит поля ввода под каждый столбец; автоинкрементный id пропускается (рисунок 16).

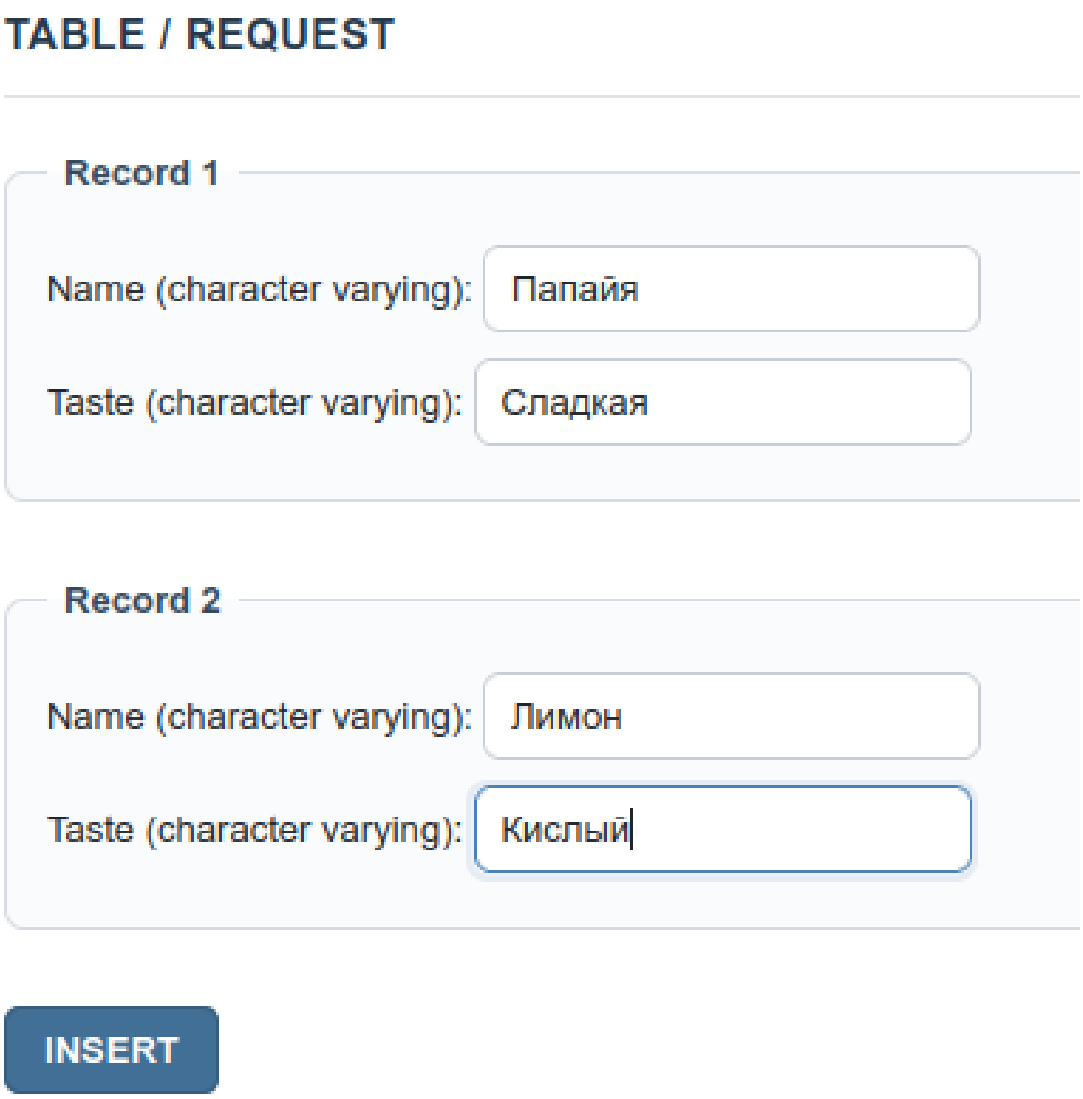


Рисунок 16 — Второй шаг вставки: заполнение полей для каждой строки

При отправке формы обработчик insert_execute формирует пакетную вставку, безопасно экранируя литералы через $pdo->quote() (листинг 6):

```
$sql = 'INSERT INTO ' . quoteIdent($table) . '(' . implode(', ',
   $columnNames) . ') VALUES ';
for ($i = 0; $i < $count; $i++) {
    $rowValues = [];
    foreach ($columns as $column) {
        $col = $column['column_name'];
```

```
        if ($col === 'id' && str_starts_with((string)$column['
   column_default'], 'nextval(')) continue;
        $rowValues[] = $pdo->quote($values[$i][$col] ?? null);
    }
    $insertRows[] = '(' . implode(', ', $rowValues) . ')';
}
$pdo->exec($sql . implode(', ', $insertRows) . ';');
```

Листинг 6 — Пакетная сборка запроса INSERT INTO

Занесем в таблицу Fruits две записи — Папайя со значением столбца Name и Лимон с аналогичным заполнением. После выполнения обе строки появляются в выборке (рисунок 17).

TABLE / REQUEST

Id	Name	Taste
1	Яблоко	*NULL*
2	Груша	*NULL*
3	Папайя	Сладкая
4	Лимон	Кислый

Рисунок 17 — Результат выполнения команды INSERT

Команда UPDATE изменяет значения указанных столбцов у строк, удовлетворяющих условию WHERE. Синтаксис: UPDATE имя SET колонка = значение WHERE колонка оператор значение. В панели форма UPDATE состоит из двух частей: блок SET (выбор столбца и новое значение) и блок WHERE (столбец, оператор и значение сравнения). Первый шаг — выбор таблицы (рисунок 18).

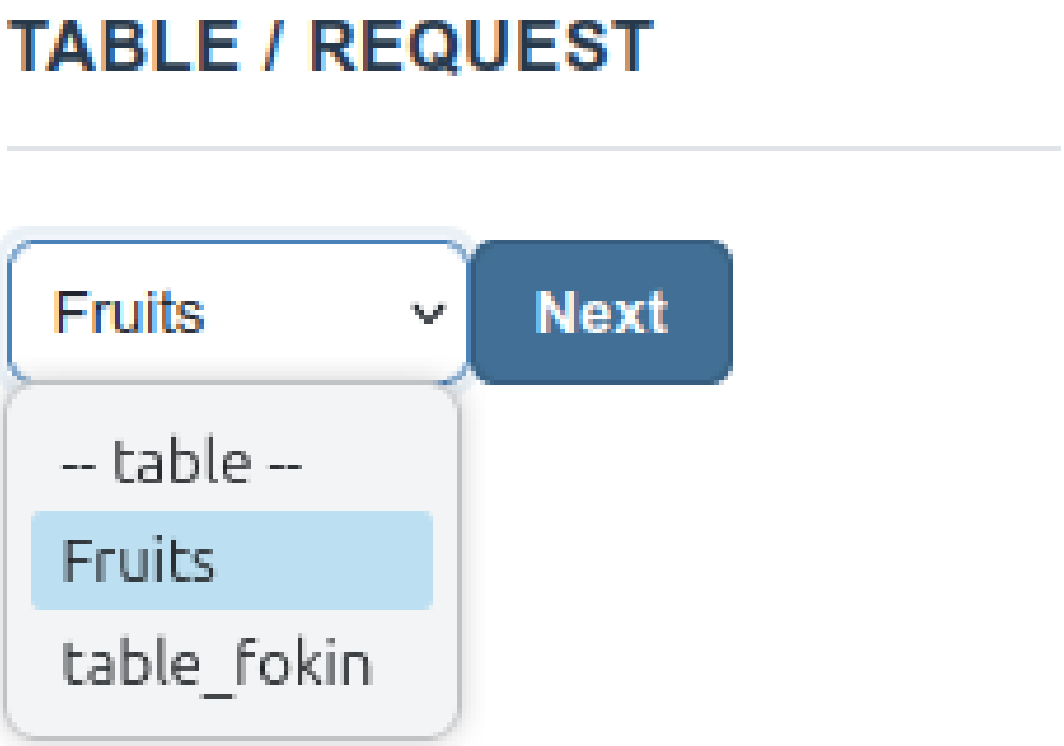


Рисунок 18 — Первый шаг UPDATE: выбор таблицы

Второй шаг — заполнение блоков SET и WHERE (рисунок 19).

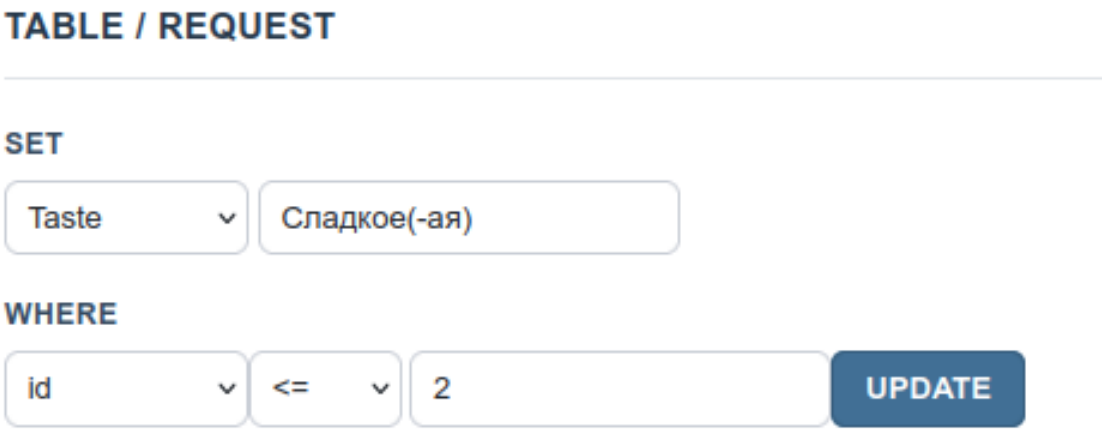


Рисунок 19 — Второй шаг UPDATE: блоки SET и WHERE

Обработчик экранирует имена колонок через quoteIdent(), а значения — через $pdo->quote() (листинг 7):

```
$sql = 'UPDATE ' . quoteIdent($table) . ' SET ' . quoteIdent(
   $setCol) . ' = ' . $pdo->quote($setVal)
```

```
. ' WHERE ' . quoteIdent($whereCol) . ' ' . $operator . ' ' .
   $pdo->quote($whereVal) . ';';
$pdo->exec($sql);
```

Листинг 7 — Сборка запроса UPDATE

Проставим для первых двух записей таблицы Fruits в столбце Taste значение "Сладкое(-ая)" с условием id <= 2. После выполнения обе строки получают одинаковое значение (рисунок 20).

TABLE / REQUEST

id	Name	Taste
3	Папайя	Сладкая
4	Лимон	Кислый
1	Яблоко	Сладкое(-ая)
2	Груша	Сладкое(-ая)

Рисунок 20 — Результат выполнения команды UPDATE

Команда DELETE удаляет строки, удовлетворяющие условию. Синтаксис: DELETE FROM имя WHERE колонка оператор значение. Форма устроена так же, как у UPDATE, но содержит только блок WHERE. Первый шаг — выбор таблицы (рисунок 21), второй — задание условия отбора (рисунок 22).

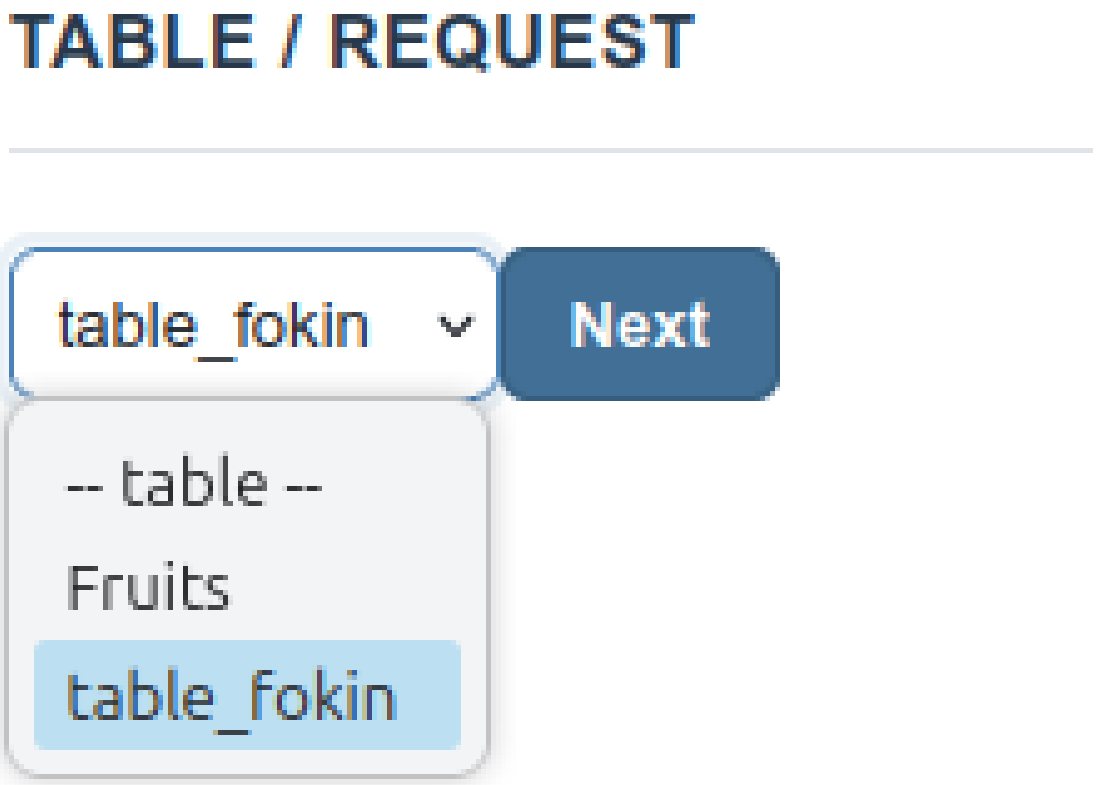


Рисунок 21 — Первый шаг DELETE: выбор таблицы



Рисунок 22 — Второй шаг DELETE: условие отбора

Серверный обработчик собирает запрос DELETE FROM "таблица" WHERE "колонка" [оператор] 'значение' (листинг 8):

```
$sql = 'DELETE FROM ' . quoteIdent($table) . ' WHERE ' .
   quoteIdent($whereCol) . ' ' . $operator .' ' . $pdo->quote(
   $whereVal) . ';';
$pdo->exec($sql);
```

Листинг 8 — Сборка запроса DELETE

Удалим из таблицы Table_Fokin всех сотрудников с зарплатой меньше 20000 (условие ЗП < 20000). Поскольку в таблице три таких записи, после выполнения операции в выборке остается 17 строк (рисунок 23).

TABLE / REQUEST

Id	Фио	Номер телефона	Адрес	ЗП	Начал работать
22	Климов Константин Кириешко	+7999223345	ул. Татарская 4	28000	2022-12-12
23	Жуков Панкратий Георгиевич	+7777123456	ул. Шнобелевская 105	55120	2023-07-02
24	Беляев Вилен Максович	+7123445590	ул. Валерьева 55	43550	2020-10-20
25	Виноградов Карл Михайлович	+79073425542	ул. Дворская 45	45800	2009-11-30
26	Захаров Витольд Фролович	+73456428800	ул. Ленина 22	99000	1990-05-06
27	Капустин Макар Валентинович	+78886547777	ул. Новодворская 47	77500	2016-09-14
28	Лобанов Андрей Гордеевич	+76665557474	ул. Куфлонина 57	34000	2026-01-02
29	Лихачёв Степан Тихонович	+71230983784	ул. Светлая 12	48753	2020-06-17
30	Пахомов Всеволод Богуславович	+78572348539	ул. Базарная 77	24231	2024-04-03
31	Бобылёв Богдан Серапионович	+72938402394	ул. Понедельная 5	88900	2026-06-12
32	Дорофеев Харитон Донатович	+72338479123	ул. Крымская 7 к. 2	100000	2011-02-22
33	Морозов Святослав Кимович	+73427283124	ул. Постная 4	23334	2023-09-10
36	Мамонтов Игнат Германнович	+78310998431	ул. Ленина 24	55890	2022-12-04
37	Фадеев Соломон Максимович	+73419010234	ул. Гагарина 89	34500	2018-02-12
38	Ковалёв Ефрем Георгьевич	+79923913219	ул. Крестьянская 8	123453	2000-01-01
39	Семёнов Соломон Сергеевич	+79231887412	ул. Фабричная 29	84231	2025-01-19
40	Мамонтов Емельян Иринеевич	+71233482134	ул. Ордынская 44	30000	2009-10-16

Рисунок 23 — Результат выполнения команды DELETE

1.4 Выборка SELECT и соединение INNER JOIN

Команда SELECT относится к группе DQL (Data Query Language), но по традиции рассматривается вместе с DML, поскольку также работает с данными, а не со структурой. Общий синтаксис: SELECT список_колонок FROM имя [WHERE условие] [ORDER BY колонка ASC|DESC].

Конструктор SELECT позволяет пользователю гибко собрать проекцию таблицы с опциональной фильтрацией и сортировкой. Первый шаг — выбор таблицы (рисунок 24).

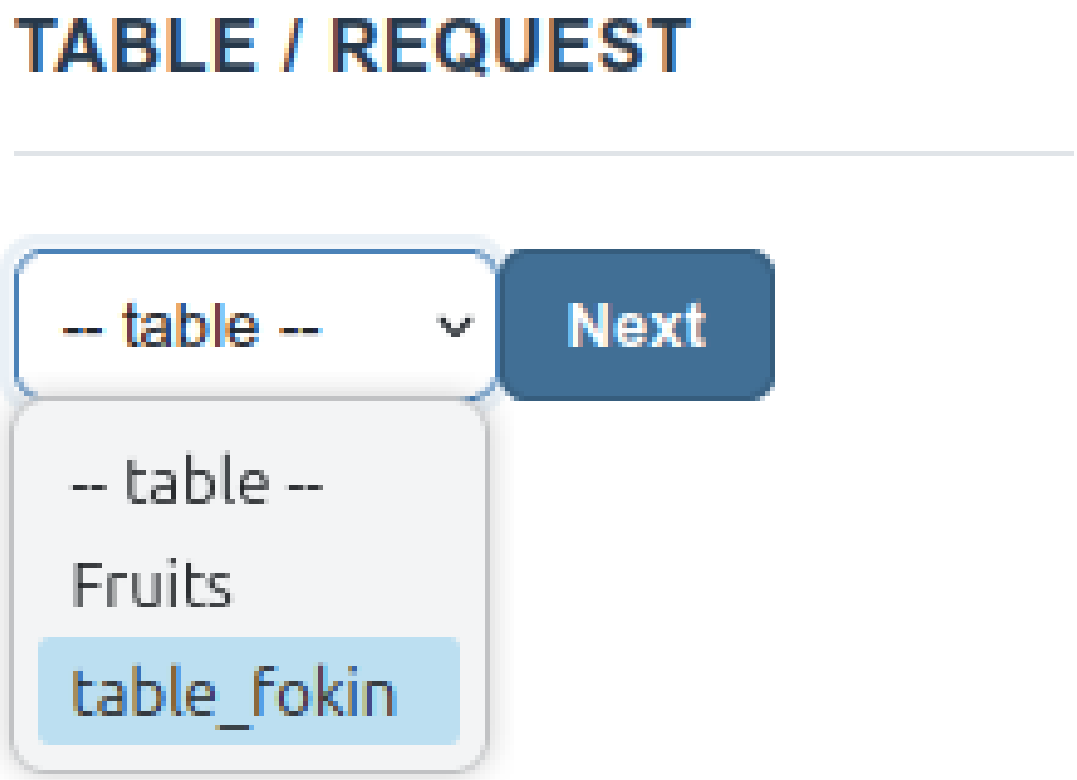


Рисунок 24 — Первый шаг SELECT: выбор таблицы

На втором шаге пользователь чекбоксами отмечает нужные столбцы (если не выбран ни один, подставляется "*"), при желании задает условие WHERE и столбец для сортировки ORDER BY с направлением ASC или DESC (рисунок 25).

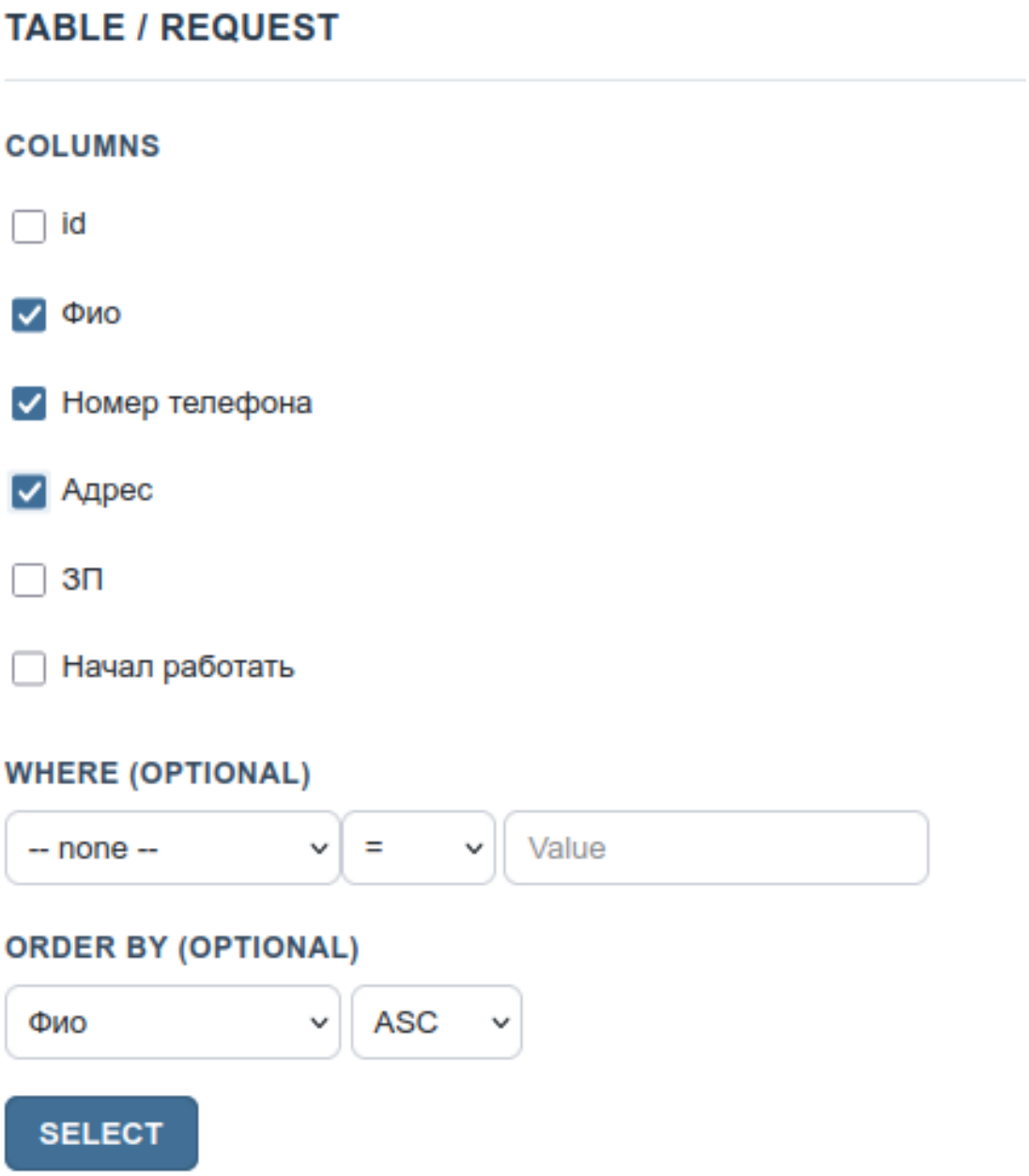


Рисунок 25 — Второй шаг SELECT: столбцы, WHERE и ORDER BY

Серверный обработчик собирает запрос по частям (листинг 9):

```
$sql = 'SELECT ' . (empty($columns) ? '*' : implode(', ',
   $columns)) . ' FROM ' . quoteIdent($table);
if ($whereCol !== '' && $whereVal !== '') {
    $sql .= ' WHERE ' . quoteIdent($whereCol) . $operator . $pdo
   ->quote($whereVal);
}
if ($order_by !== '') {
    $sql .= ' ORDER BY ' . quoteIdent($order_by) . ' ' . h(
   $order_how);
}
```

```
$rows = $pdo->query($sql . ';')->fetchAll(PDO::FETCH_ASSOC);
```

Листинг 9 — Сборка сложного запроса SELECT

Выполним выборку колонок Фио, "Номер телефона" и Адрес из таблицы Table_Fokin с сортировкой по Фио по возрастанию. Запрос возвращает упорядоченный набор данных (рисунок 26).

TABLE / REQUEST

Фио	Номер телефона	Адрес
Беляев Вилен Максович	+7123445590	ул. Валерьева 55
Бобылёв Богдан Серапионович	+72938402394	ул. Понедельная 5
Виноградов Карл Михайлович	+79073425542	ул. Дворская 45
Дорофеев Харитон Донатович	+72338479123	ул. Крымская 7 к. 2
Жуков Панкратий Георгиевич	+7777123456	ул. Шнобелевская 105
Захаров Витольд Фролович	+73456428800	ул. Ленина 22
Капустин Макар Валентинович	+78886547777	ул. Новодворская 47
Климов Константин Кириешко	+7999223345	ул. Татарская 4
Ковалёв Ефрем Георгьевич	+79923913219	ул. Крестьянская 8
Лихачёв Степан Тихонович	+71230983784	ул. Светлая 12
Лобанов Андрей Гордеевич	+76665557474	ул. Куфлонина 57
Мамонтов Емельян Иринеевич	+71233482134	ул. Ордынская 44
Мамонтов Игнат Германнович	+78310998431	ул. Ленина 24
Морозов Святослав Кимович	+73427283124	ул. Постная 4
Пахомов Всеволод Богуславович	+78572348539	ул. Базарная 77
Семёнов Соломон Сергеевич	+79231887412	ул. Фабричная 29
Фадеев Соломон Максимович	+73419010234	ул. Гагарина 89

Рисунок 26 — Результат выполнения команды SELECT

Оператор INNER JOIN объединяет строки двух таблиц по условию равенства указанных колонок; в результирующий набор попадают только те пары строк, для которых условие выполнено. Синтаксис: SELECT список_колонок FROM таблица1 INNER JOIN таблица2 ON таблица1.колонка = таблица2.колонка.

Для демонстрации соединения заранее подготовим небольшую вспомогательную таблицу Mobile_operators, в которой для пяти телефонных номеров из Table_Fokin указан оператор сотовой связи. Первый шаг конструктора — выбор двух таблиц из базы (рисунок 27).

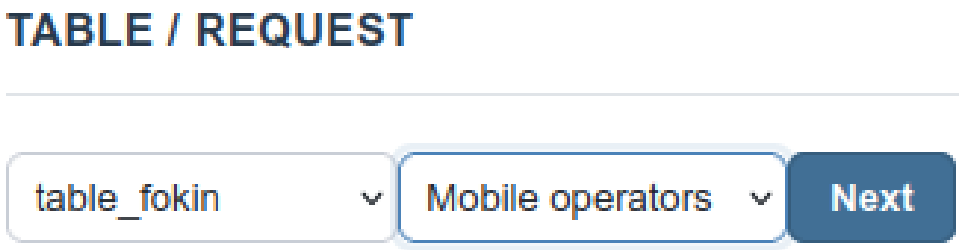


Рисунок 27 — Первый шаг INNER JOIN: выбор двух таблиц

Второй шаг совмещает два действия: выбор связующих колонок для условия ON и выбор выводимых столбцов для SELECT. Список столбцов первой таблицы отображается первым, второй — вторым; нужные поля отмечаются чекбоксами. Если не отмечен ни один столбец, в выборку попадут все поля обеих таблиц. Функция, генерирующая форму второго шага, приведена в листинге 10, внешний вид формы — на рисунке 28.

```
function joinWorkspace($pdo) {
    $table1 = post('table1');
    $table2 = post('table2');
    if ($table1 === '' || $table2 === '') {
        echo '<form method="post">';
        echo tableSelect($pdo, 'table1', $table1);
        echo tableSelect($pdo, 'table2', $table2);
        echo button('join_form', 'Next');
        echo '</form>';
        return;
    }

    $columns1 = getColumns($pdo, $table1);
    $columns2 = getColumns($pdo, $table2);
    echo '<form method="post">';
    echo '<input type="hidden" name="table1" value="' . h($table1
) . '">';
    echo '<input type="hidden" name="table2" value="' . h($table2
) . '">';

    echo '<h4>COLUMNS (First table)</h4>';
```

```
    foreach ($columns1 as $column1) {
        $name = $column1['column_name'];
        echo '<label><input type="checkbox" name="selected_columns1[]" value="' . h($name) . '"> ' . h($name) . '</label><br>';
    }

    echo '<h4>COLUMNS (Second table)</h4>';
    foreach ($columns2 as $column2) {
        $name = $column2['column_name'];
        echo '<label><input type="checkbox" name="selected_columns2[]" value="' . h($name) . '"> ' . h($name) . '</label><br>';
    }

    echo '<p>' . h($table1) . ': ' . columnSelect($columns1, 'column1') . '</p>';
    echo '<p>' . h($table2) . ': ' . columnSelect($columns2, 'column2') . '</p>';
    echo button('join_execute', 'INNER JOIN');
    echo '</form>';
}
```

Листинг 10 — Форма второго шага INNER JOIN

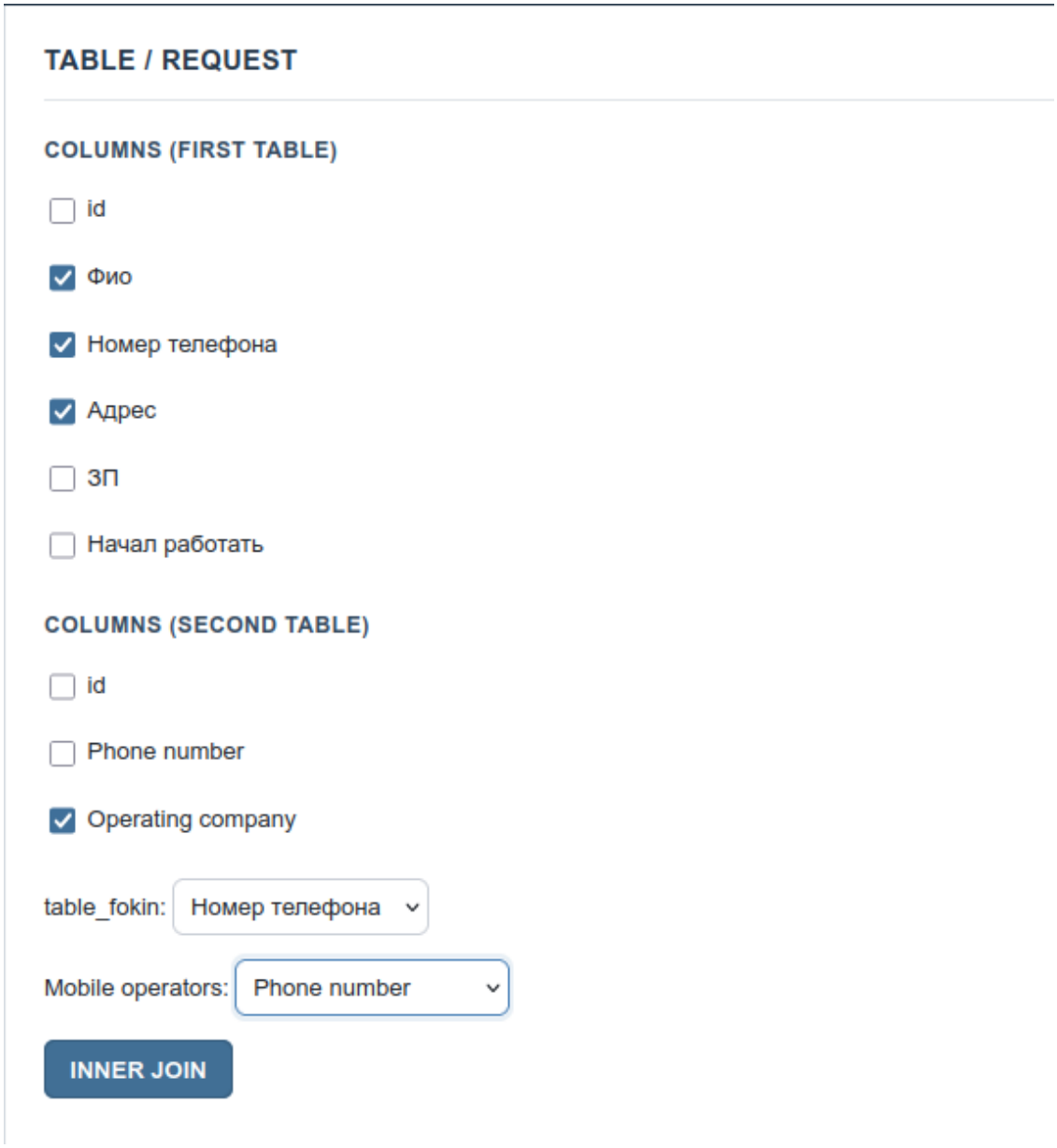


Рисунок 28 — Второй шаг INNER JOIN: столбцы и связующие колонки

Серверный обработчик собирает список колонок из отмеченных чекбоксов, добавляя каждому имени префикс соответствующей таблицы, и подставляет готовый список в SELECT (листинг 11):

```
elseif ($action === 'join_execute') {
    $table1 = post('table1');
    $table2 = post('table2');
    $from = post('column1');
    $to = post('column2');
    $pks1 = post('selected_columns1', []);
    $pks2 = post('selected_columns2', []);

    if ($table1 === '' || $table2 === '') {
```

```
        $error = 'No table selected.';
    } elseif (!in_array($table1, getTables($pdo), true) || !in_array($table2, getTables($pdo), true)) {
        $error = 'Table does not exist.';
    } else {
        try {
            $allColumns1 = getColumns($pdo, $table1);
            $allColumns2 = getColumns($pdo, $table2);
            $columns = [];
            foreach ($allColumns1 as $col1) {
                if (in_array($col1['column_name'], $pks1, true)) {
                    $columns[] = quoteIdent($table1) . '.' . quoteIdent($col1['column_name']);
                }
            }
            foreach ($allColumns2 as $col2) {
                if (in_array($col2['column_name'], $pks2, true)) {
                    $columns[] = quoteIdent($table2) . '.' . quoteIdent($col2['column_name']);
                }
            }
            $sql = 'SELECT ' . (empty($columns) ? '*' : implode(', ', $columns)) . ' FROM ' . quoteIdent($table1) . ' INNER JOIN ' . quoteIdent($table2)
                . ' ON ' . quoteIdent($table1) . '.' . quoteIdent($from) . ' = ' . quoteIdent($table2) . '.' . quoteIdent($to) . ';';
            $rows = $pdo->query($sql)->fetchAll(PDO::FETCH_ASSOC);
        } catch (PDOException $e) {
            $error = 'Join failed: ' . h($e->getMessage());
        }
    }
}
```

Листинг 11 — Формирование запроса INNER JOIN

Выполним соединение таблиц Table_Fokin и Mobile_operators по номеру телефона. Из первой таблицы выберем Фио, Номер телефона и Адрес, из второй — Operating company (рисунок 29).

TABLE / REQUEST

Фио	Номер телефона	Адрес	Operating company
Климов Константин Кириешко	+7999223345	ул. Татарская 4	Ростелеком
Виноградов Карл Михайлович	+79073425542	ул. Дворская 45	t2
Лобанов Андрей Гордеевич	+76665557474	ул. Куфлонина 57	Билайн
Дорофеев Харитон Донатович	+72338479123	ул. Крымская 7 к. 2	t2
Ковалёв Ефрем Георгьевич	+79923913219	ул. Крестьянская 8	МТС

Рисунок 29 — Результат выполнения запроса INNER JOIN

В результирующей таблице четыре столбца — ровно те, что были отмечены. Колонка Номер телефона, по которой выполняется соединение, присутствует один раз, так как взята только из Table_Fokin. При использовании SELECT * она дублировалась бы по одному экземпляру из каждой таблицы; явный список колонок устраняет это дублирование.

Делая выводы, была спроектирована и реализована полноценная веб-панель управления базой данных на PHP с использованием PostgreSQL. Все команды DDL и DML, а также соединение INNER JOIN динамически формируются на основе действий пользователя, корректно экранируются и выводят результаты в браузер. Изоляция изменений в отдельной базе данных db4 позволила свободно тестировать операции удаления и модификации схемы без риска повреждения данных предыдущих работ.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и практически реализованы на веб-странице основные команды языка SQL групп DDL и DML, а также оператор соединения таблиц INNER JOIN:

1) Создана отдельная база данных db4 для безопасного тестирования мутирующих команд;
2) Освоены команды DDL (CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE TABLE) для управления структурой базы данных через интерфейс браузера;
3) Освоены команды DML (INSERT INTO, UPDATE, DELETE, SELECT) для модификации и выборки строк;
4) Реализовано динамическое формирование SQL-запросов на основе параметров $_POST с экранированием данных через функции драйвера PDO;
5) Продемонстрирована работа операции INNER JOIN для объединения кортежей из нескольких связанных таблиц;
6) Все сформированные выборки и результаты выполнения команд успешно выведены на страницу браузера.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Дейт, К. Дж. Введение в системы баз данных : учебный курс : [перевод с английского] / К. Дж. Дейт. — 8-е изд. — Москва ; Санкт-Петербург ; Киев : Вильямс, 2019. — 1327 с. — ISBN 978-5-8459-0788-2. — Текст : непосредственный.
2. Хомоненко, А. Д. Базы данных : учебник для вузов / А. Д. Хомоненко, В. М. Цыганков, М. Г. Мальцев ; под редакцией А. Д. Хомоненко. — 6-е изд., сокр. и дораб. — Санкт-Петербург : Корона-Век, 2021. — 496 с. — ISBN 978-5-7931-1052-5. — Текст : непосредственный.
3. Грубер, М. Понимание SQL : [перевод с английского] / Мартин Грубер. — Москва : Лаборатория знаний, 2022. — 352 с. — ISBN 978-5-00101-925-1. — Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Полный исходный код скрипта панели управления базой данных

```php
<?php
error_reporting(E_ALL);
ini_set('display_errors', '1');

$host = 'localhost';
$dbname = 'db4';
$user = 'postgres';
$password = 'PASSWORD';

try {
    $pdo = new PDO("pgsql:host=$host;dbname=$dbname", $user, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die('Database connection failed: ' . h($e->getMessage()));
}

function h($value) {
    return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}

function post($name, $default = '') {
    return $_POST[$name] ?? $default;
}

function button($action, $text) {
    return '<button type="submit" name="action" value="' . h($action) . '">' . h($text) . '</button>';
}

function formButton($action, $text) {
    return '<form method="post" style="display:inline">' . button($action, $text) . '</form>';
}

```

```php
function quoteIdent($name) {
    return '"' . str_replace('"', '""', $name) . '"';
}

function getTables($pdo) {
    $sql = "
    SELECT table_name
    FROM information_schema.tables
    WHERE table_schema = 'public'
    AND table_type = 'BASE TABLE'
    ORDER BY table_name
    ";
    return $pdo->query($sql)->fetchAll(PDO::FETCH_COLUMN);
}

function getColumns($pdo, $table) {
    $sql = "
    SELECT column_name, data_type, column_default
    FROM information_schema.columns
    WHERE table_schema = 'public'
    AND table_name = :table
    ORDER BY ordinal_position
    ";
    $stmt = $pdo->prepare($sql);
    $stmt->execute(['table' => $table]);
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

function tableSelect($pdo, $name = 'table', $selected = '') {
    $html = '<select name="' . h($name) . '" required>';
    $html .= '<option value="">-- table --</option>';
    foreach (getTables($pdo) as $table) {
        $mark = $table === $selected ? ' selected' : '';
        $html .= '<option value="' . h($table) . '"' . $mark . '>
   ' . h($table) . '</option>';
    }
    return $html . '</select>';
}

function columnSelect($columns, $name, $selected = '', $optional
   = false) {
```

```
    $html = '<select name="' . h($name) . '"' . ($optional ? '' : ' required') . '>';
    $html .= $optional ? '<option value="">-- none --</option>' : '<option value="">-- column --</option>';
    foreach ($columns as $column) {
        $nameValue = $column['column_name'];
        $mark = $nameValue === $selected ? ' selected' : '';
        $html .= '<option value="' . h($nameValue) . '"' . $mark . '>' . h($nameValue) . '</option>';
    }
    return $html . '</select>';
}

function operatorSelect($name = 'operator') {
    $operators = ['=', '<>', '<', '>', '<=', '>=', 'LIKE'];
    $html = '<select name="' . h($name) . '">';
    foreach ($operators as $operator) {
        $html .= '<option value="' . h($operator) . '">' . h($operator) . '</option>';
    }
    return $html . '</select>';
}

function createWorkspace() {
    echo '<form method="post">';
    echo '<input name="table_name" placeholder="Table name" required> ';
    echo '<input type="number" name="column_count" min="1" value="1" required> ';
    echo button('create_columns', 'Next');
    echo '</form>';
    echo '<p>Enter table name and number of columns.</p>';
}

function createColumnsWorkspace() {
    $table = post('table_name');
    $count = max(1, (int)post('column_count', 1));
    $types = ['INTEGER', 'BIGINT', 'NUMERIC', 'VARCHAR(255)', 'TEXT', 'DATE', 'BOOLEAN'];

    echo '<form method="post">';
```

```
    echo '<input type="hidden" name="table_name" value="' . h($table) . '">';
    echo '<input type="hidden" name="column_count" value="' . $count . '">';
    echo '<table class="data-table" width="100%">';
    echo '<tr><th>#</th><th>Name</th><th>Type</th><th>PK</th></tr>';

    for ($i = 0; $i < $count; $i++) {
        echo '<tr>';
        echo '<td>' . ($i + 1) . '</td>';
        echo '<td><input name="column_name[' . $i . ']" required></td>';
        echo '<td><select name="data_type[' . $i . ']">';
        foreach ($types as $type) {
            echo '<option value="' . h($type) . '">' . h($type) . '</option>';
        }
        echo '</select></td>';
        echo '<td><input type="checkbox" name="primary_key[' . $i . ']" value="1"></td>';
        echo '</tr>';
    }

    echo '</table><br>';
    echo button('create_execute', 'CREATE');
    echo '</form>';
    echo '<p>If no PK is checked, add the automatic <code>id</code> identity PK in your CREATE query.</p>';
}

function alterWorkspace($pdo) {
    $types = ['INTEGER', 'BIGINT', 'NUMERIC', 'VARCHAR(255)', 'TEXT', 'DATE', 'BOOLEAN'];
    echo '<form method="post">';
    echo tableSelect($pdo);
    echo '<select name="alter_type">';
    echo '<option value="add">ADD COLUMN</option>';
    echo '<option value="alter">ALTER COLUMN</option>';
    echo '<option value="drop">DROP COLUMN</option>';
    echo '</select> ';
```

```
    echo '<input name="column_name" placeholder="Column name"> ';
    echo '<select name="value">';
    foreach ($types as $type) {
        echo '<option value="' . h($type) . '">' . h($type) . '</option>';
    }
    echo '</select>';
    echo button('alter_execute', 'ALTER');
    echo '</form>';
}

function simpleTableWorkspace($pdo, $action, $label) {
    echo '<form method="post">';
    echo tableSelect($pdo);
    echo button($action . '_execute', $label);
    echo '</form>';
}

function insertWorkspace($pdo) {
    $table = post('table');
    $count = max(1, (int)post('row_count', 1));

    echo '<form method="post">';
    echo tableSelect($pdo, 'table', $table);
    echo '<input type="number" name="row_count" min="1" value="' . $count . '" required> ';
    echo button('insert_rows', 'Next');
    echo '</form>';
}

function insertRowsWorkspace($pdo) {
    $table = post('table');
    $count = max(1, (int)post('row_count', 1));
    $columns = getColumns($pdo, $table);

    echo '<form method="post">';
    echo '<input type="hidden" name="table" value="' . h($table) . '">';
    echo '<input type="hidden" name="row_count" value="' . $count . '">';

```

```
    for ($row = 0; $row < $count; $row++) {
        echo '<fieldset><legend>Record ' . ($row + 1) . '</legend>';
        foreach ($columns as $column) {
            $name = $column['column_name'];
            if ($name === 'id' && str_starts_with((string)$column['column_default'], 'nextval(')) continue;
            echo '<label>' . h($name) . ' (' . h($column['data_type']) . '): ';
            echo '<input name="values[' . $row . '][' . h($name) . ']">';
            echo '</label><br>';
        }
        echo '</fieldset><br>';
    }
    echo button('insert_execute', 'INSERT');
    echo '</form>';
}

function wherePart($columns) {
    echo '<h4>WHERE</h4>';
    echo columnSelect($columns, 'where_column');
    echo operatorSelect();
    echo ' <input name="where_value" placeholder="Value" required>';
}

function updateWorkspace($pdo) {
    $table = post('table');
    if ($table === '') {
        echo '<form method="post">';
        echo tableSelect($pdo);
        echo button('update_form', 'Next');
        echo '</form>';
        return;
    }

    $columns = getColumns($pdo, $table);
    echo '<form method="post">';
    echo '<input type="hidden" name="table" value="' . h($table) . '">';
```

```
211     echo '<h4>SET</h4>';
212     echo columnSelect($columns, 'set_column');
213     echo ' <input name="set_value" placeholder="New value"
        required>';
214     wherePart($columns);
215     echo button('update_execute', 'UPDATE');
216     echo '</form>';
217 }
218 
219 function deleteWorkspace($pdo) {
220     $table = post('table');
221     if ($table === '') {
222         echo '<form method="post">';
223         echo tableSelect($pdo);
224         echo button('delete_form', 'Next');
225         echo '</form>';
226         return;
227     }
228 
229     $columns = getColumns($pdo, $table);
230     echo '<form method="post">';
231     echo '<input type="hidden" name="table" value="' . h($table)
        . '">';
232     wherePart($columns);
233     echo button('delete_execute', 'DELETE');
234     echo '</form>';
235 }
236 
237 function selectWorkspace($pdo) {
238     $table = post('table');
239     if ($table === '') {
240         echo '<form method="post">';
241         echo tableSelect($pdo);
242         echo button('select_form', 'Next');
243         echo '</form>';
244         return;
245     }
246 
247     $columns = getColumns($pdo, $table);
248     echo '<form method="post">';
249     echo '<input type="hidden" name="table" value="' . h($table)
```

```
    . '">';
    echo '<h4>COLUMNS</h4>';
    foreach ($columns as $column) {
        $name = $column['column_name'];
        echo '<label><input type="checkbox" name="
    selected_columns[]" value="' . h($name) . '"> ' . h($name) . '
    </label><br>';
    }
    echo '<h4>WHERE (optional)</h4>';
    echo columnSelect($columns, 'where_column', '', true);
    echo operatorSelect();
    echo ' <input name="where_value" placeholder="Value">';
    echo '<h4>ORDER BY (optional)</h4>';
    echo columnSelect($columns, 'order_column', '', true);
    echo ' <select name="order_direction"><option>ASC</option><
    option>DESC</option></select>';
    echo '<br><br>' . button('select_execute', 'SELECT');
    echo '</form>';
}

function joinWorkspace($pdo) {
    $table1 = post('table1');
    $table2 = post('table2');
    if ($table1 === '' || $table2 === '') {
        echo '<form method="post">';
        echo tableSelect($pdo, 'table1', $table1);
        echo tableSelect($pdo, 'table2', $table2);
        echo button('join_form', 'Next');
        echo '</form>';
        return;
    }

    $columns1 = getColumns($pdo, $table1);
    $columns2 = getColumns($pdo, $table2);
    echo '<form method="post">';
    echo '<input type="hidden" name="table1" value="' . h($table1
    ) . '">';
    echo '<h4>COLUMNS (First table)</h4>';
    foreach ($columns1 as $column1) {
        $name = $column1['column_name'];
        echo '<label><input type="checkbox" name="
```

```
    selected_columns1[]" value="' . h($name) . '"> ' . h($name) .
    '</label><br>';
286      }
287      echo '<input type="hidden" name="table2" value="' . h($table2
    ) . '">';
288      echo '<h4>COLUMNS (Second table)</h4>';
289      foreach ($columns2 as $column2) {
290          $name = $column2['column_name'];
291          echo '<label><input type="checkbox" name="
    selected_columns2[]" value="' . h($name) . '"> ' . h($name) .
    '</label><br>';
292      };
293      echo '<input type="hidden" name="table1" value="' . h($table1
    ) . '">';
294      echo '<input type="hidden" name="table2" value="' . h($table2
    ) . '">';
295      echo '<p>' . h($table1) . ': ' . columnSelect($columns1, '
    column1') . '</p>';
296      echo '<p>' . h($table2) . ': ' . columnSelect($columns2, '
    column2') . '</p>';
297      echo button('join_execute', 'INNER JOIN');
298      echo '</form>';
299  }
300
301  function sendTable($rows) {
302      if (!empty($rows)) {
303          echo '<table class="data-table">';
304          echo '<thead><tr>';
305          foreach (array_keys($rows[0]) as $columnName) {
306              echo '<th>' . htmlspecialchars($columnName) . '</th>'
    ;
307          }
308          echo '</tr></thead><tbody>';
309          foreach ($rows as $row) {
310              echo '<tr>';
311              foreach ($row as $value) {
312                  echo '<td>' . (($value !== null) ?
    htmlspecialchars($value) : "<i>NULL</i>") . '</td>';
313              }
314              echo '</tr>';
315          }
```

```
316         echo '</tbody></table>';
317     } else {
318         echo '<p class="msg-emptyЗаписей">, соответствующих
    данному запросу, не найдено или таблица пуста.</p>';
319     }
320 }
321 
322 $action = post('action', 'show');
323 $rows = [];
324 $allRows = [];
325 
326 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
327     if ($action === 'show_execute') {
328         $table = post('table');
329         if ($table === '') {
330             $error = 'No table name.';
331         } elseif (!in_array($table, getTables($pdo), true)) {
332             $error = 'No table in the database.';
333         } else {
334             $sql = 'SELECT * FROM ' . quoteIdent($table) . ';';
335             $rows = $pdo->query($sql)->fetchAll(PDO::FETCH_ASSOC)
    ;
336         }
337     }
338     elseif ($action === 'show_all') {
339         $tables = getTables($pdo);
340         if (empty($tables)) {
341             $error = 'No tables.';
342         } else {
343             foreach ($tables as $table) {
344                 $sql = 'SELECT * FROM ' . quoteIdent($table) . ';
    ';
345                 $allRows[$table] = $pdo->query($sql)->fetchAll(
    PDO::FETCH_ASSOC);
346             }
347         }
348     }
349     elseif ($action === 'create_execute') {
350         $table = post('table_name');
351         if ($table === '') {
352             $error = 'No table name.';
```

```
        } elseif (in_array($table, getTables($pdo), true)) {
            $error = 'Table already exists.';
        } else {
            $count = (int) post('column_count', 1);
            $names = post('column_name', []);
            $types = post('data_type', []);
            $pks   = post('primary_key', []);
            $args  = [];

            if (empty($pks)) {
                $args[] = 'id SERIAL PRIMARY KEY';
            }

            for ($i = 0; $i < $count; $i++) {
                $argName = $names[$i] ?? "";
                $argType = $types[$i] ?? "";

                $arg = quoteIdent($argName) . ' ' . $argType;
                if (!empty($pks[$i])) {
                    $arg .= ' PRIMARY KEY';
                }
                $args[] = $arg;
            }

            $sql = 'CREATE TABLE ' . quoteIdent($table) . '(' .
implode(', ', $args) . ');';
            try {
                $pdo->exec($sql);
                $message = 'Table ' . h($table) . ' created
successfully';
            } catch (PDOException $e) {
                $error = 'Create failed: ' . h($e->getMessage());
            }
        }
    }
    elseif ($action === 'alter_execute') {
        $table = post('table');
        $alter_type = post('alter_type');
        $newColName = post('column_name');
        $newColType = post('value');
        if ($table === '') {
```

```
            $error = 'No table selected.';
        } elseif (!in_array($table, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            $sql = 'ALTER TABLE ' . quoteIdent($table) . ' ';
            try {
                if ($alter_type === 'add') {
                    $sql .= 'ADD ' . quoteIdent($newColName) . '
' . $newColType;
                } elseif ($alter_type === 'alter') {
                    $sql .= 'ALTER COLUMN ' . quoteIdent(
$newColName) . ' TYPE ' . $newColType;
                } elseif ($alter_type === 'drop') {
                    $sql .= 'DROP COLUMN ' . quoteIdent(
$newColName);
                } else {
                    throw new PDOException('Unknown alter type');
                }

                $pdo->exec($sql . ';');
                $message = 'Table ' . h($table) . ' altered
successfully';
            } catch (PDOException $e) {
                $error = 'Alter failed: ' . h($e->getMessage());
            }
        }
     }
     elseif ($action === 'drop_execute') {
        $table = post('table');
        if ($table === '') {
            $error = 'No table selected.';
        } elseif (!in_array($table, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            try {
                $pdo->exec('DROP TABLE ' . quoteIdent($table));
                $message = 'Table ' . h($table) . ' dropped
successfully.';
            } catch (PDOException $e) {
                $error = 'Drop failed: ' . h($e->getMessage());
            }
```

```
        }
    }
    elseif ($action === 'truncate_execute') {
        $table = post('table');
        if ($table === '') {
            $error = 'No table selected.';
        } elseif (!in_array($table, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            try {
                $pdo->exec('TRUNCATE TABLE ' . quoteIdent($table)
);
                $message = 'Table ' . h($table) . ' truncated
successfully.';
            } catch (PDOException $e) {
                $error = 'Truncate failed: ' . h($e->getMessage()
);
            }
        }
    }
    elseif ($action === 'insert_execute') {
        $table = post('table');
        $count = (int) post('row_count', 1);
        $values = post('values', []);

        if ($table === '') {
            $error = 'No table selected.';
        } elseif (!in_array($table, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            try {
                $columns = getColumns($pdo, $table);
                $columnNames = [];
                foreach ($columns as $column) {
                    if ($column['column_name'] === 'id' &&
str_starts_with((string)$column['column_default'], 'nextval(')
) continue;
                    $columnNames[] = quoteIdent($column['
column_name']);
                }

```

```
                $sql = 'INSERT INTO ' . quoteIdent($table) . '(' . implode(', ', $columnNames) . ') VALUES ';
                $insertRows = [];

                for ($i = 0; $i < $count; $i++) {
                    $rowValues = [];
                    foreach ($columns as $column) {
                        $col = $column['column_name'];
                        if ($col === 'id' && str_starts_with((string)$column['column_default'], 'nextval(')) continue;

                        $value = $values[$i][$col] ?? null;
                        $rowValues[] = $pdo->quote($value);
                    }
                    $insertRows[] = '(' . implode(', ', $rowValues) . ')';
                }

                $pdo->exec($sql . implode(', ', $insertRows) . ';');
                $message = 'Table ' . h($table) . ' inserted into successfully.';
            } catch (PDOException $e) {
                $error = 'Insert failed: ' . h($e->getMessage());
            }
        }
    }
    elseif ($action === 'update_execute') {
        $table = post('table');
        $setCol = post('set_column');
        $setVal = post('set_value');
        $whereCol = post('where_column');
        $whereVal = post('where_value');
        $operator = post('operator');

        if ($table === '') {
            $error = 'No table selected.';
        } elseif (!in_array($table, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            $sql = 'UPDATE ' . quoteIdent($table) . ' SET ' .
```

```
quoteIdent($setCol) . ' = ' . $pdo->quote($setVal)
499             . ' WHERE ' . quoteIdent($whereCol) . ' ' . $operator
 . ' ' . $pdo->quote($whereVal) . ';';
500             try {
501                 $pdo->exec($sql);
502                 $message = 'Table ' . h($table) . ' updated
successfully.';
503             } catch (PDOException $e) {
504                 $error = 'Update failed: ' . h($e->getMessage());
505             }
506         }
507     }
508     elseif ($action === 'delete_execute') {
509         $table = post('table');
510         $whereCol = post('where_column');
511         $whereVal = post('where_value');
512         $operator = post('operator');
513 
514         if ($table === '') {
515             $error = 'No table selected.';
516         } elseif (!in_array($table, getTables($pdo), true)) {
517             $error = 'Table does not exist.';
518         } else {
519             $sql = 'DELETE FROM ' . quoteIdent($table) . ' WHERE
' . quoteIdent($whereCol) . ' ' . $operator .' ' . $pdo->quote
($whereVal) . ';';
520             try {
521                 $pdo->exec($sql);
522                 $message = 'Table ' . h($table) . ' deleted
successfully.';
523             } catch (PDOException $e) {
524                 $error = 'Deleted failed: ' . h($e->getMessage())
;
525             }
526         }
527     }
528     elseif ($action === 'select_execute') {
529         $table = post('table');
530         $whereCol = post('where_column');
531         $whereVal = post('where_value');
532         $operator = post('operator');
```

```
        $order_by = post('order_column');
        $order_how = post('order_direction');
        $pks = post('selected_columns', []);

        if ($table === '') {
            $error = 'No table selected.';
        } elseif (!in_array($table, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            try {
                $allColumns = getColumns($pdo, $table);
                $columns = [];
                foreach ($allColumns as $col) {
                    if (in_array($col['column_name'], $pks, true)) {
                        $columns[] = quoteIdent($col['column_name']);
                    }
                }
                $sql = 'SELECT ' . (empty($columns) ? '*' : implode(', ', $columns)) . ' FROM ' . quoteIdent($table);

                if ($whereCol !== '' && $whereVal !== '') {
                    $sql .= ' WHERE ' . quoteIdent($whereCol) . $operator . $pdo->quote($whereVal);
                }
                if ($order_by !== '') {
                    $sql .= ' ORDER BY ' . quoteIdent($order_by) . ' ' . h($order_how);
                }

                $rows = $pdo->query($sql . ';')->fetchAll(PDO::FETCH_ASSOC);
            } catch (PDOException $e) {
                $error = 'Select failed: ' . h($e->getMessage());
            }
        }
    }
    elseif ($action === 'join_execute') {
        $table1 = post('table1');
        $table2 = post('table2');
```

```
        $from = post('column1');
        $to = post('column2');
        $pks1 = post('selected_columns1', []);
        $pks2 = post('selected_columns2', []);

        if ($table1 === '' || $table2 === '') {
            $error = 'No table selected.';
        } elseif (!in_array($table1, getTables($pdo), true) || !in_array($table2, getTables($pdo), true)) {
            $error = 'Table does not exist.';
        } else {
            try {
                $allColumns1 = getColumns($pdo, $table1);
                $allColumns2 = getColumns($pdo, $table2);
                $columns = [];
                foreach ($allColumns1 as $col1) {
                    if (in_array($col1['column_name'], $pks1, true)) {
                        $columns[] = quoteIdent($table1) . '.' . quoteIdent($col1['column_name']);
                    }
                }
                foreach ($allColumns2 as $col2) {
                    if (in_array($col2['column_name'], $pks2, true)) {
                        $columns[] = quoteIdent($table2) . '.' . quoteIdent($col2['column_name']);
                    }
                }
                $sql = 'SELECT ' . (empty($columns) ? '*' : implode(', ', $columns)) . ' FROM ' . quoteIdent($table1) . ' INNER JOIN ' . quoteIdent($table2)
                . ' ON ' . quoteIdent($table1) . '.' . quoteIdent($from) . ' = ' . quoteIdent($table2) . '.' . quoteIdent($to) . ';';
                $rows = $pdo->query($sql)->fetchAll(PDO::FETCH_ASSOC);
            } catch (PDOException $e) {
                $error = 'Join failed: ' . h($e->getMessage());
            }
        }
```

```
    }
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Database Assignment</title>
<style>
* { box-sizing: border-box; }
body { margin: 0; padding: 30px; background: #f2f4f7; color: #20252b; font-family: Arial, sans-serif; font-size: 15px; }

.layout-container {
    display: grid;
    grid-template-columns: 220px 1fr 220px;
    grid-template-rows: auto 1fr;
    max-width: 1400px;
    margin: 0 auto;
    background: white;
    border: 1px solid #d7dce2;
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 4px 18px rgba(0, 0, 0, 0.07);
    min-height: 600px;
}

.top-nav {
    grid-column: 1 / -1;
    background: #26384a;
    padding: 14px 16px;
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
}

.sidebar { background: #f8f9fb; padding: 24px 20px; }
.sidebar-left { border-right: 1px solid #d7dce2; }
```

```
.sidebar-right { border-left: 1px solid #d7dce2; }

.sidebar b {
    display: block; margin-bottom: 18px; padding-bottom: 10px;
    border-bottom: 2px solid #dce2e8; color: #26384a;
    font-size: 17px; letter-spacing: 0.5px;
}

.workspace { padding: 24px; }
.workspace > b { display: block; color: #26384a; font-size: 17px;
    letter-spacing: 0.5px; }

form { margin: 8px 0; }
.top-nav form { margin: 0; }

input, select {
    min-height: 36px; padding: 7px 10px; border: 1px solid #
   c9ced6;
    border-radius: 6px; background: white; color: #20252b;
    font: inherit; outline: none; transition: 0.15s;
}
input:focus, select:focus { border-color: #4b82b8; box-shadow: 0
   0 0 3px rgba(75,130,184,0.12); }
input[type="checkbox"] { width: 16px; height: 16px; accent-color:
    #3f6f99; vertical-align: middle; }
input[type="number"] { width: 100px; }

button {
    min-height: 36px; padding: 7px 15px; border: 1px solid #365
   d7d;
    border-radius: 6px; background: #416f95; color: white;
    font: inherit; font-weight: 600; cursor: pointer; transition:
    0.15s;
}
button:hover { background: #345d7d; }
button:active { transform: translateY(1px); }

.top-nav button { background: #edf2f6; color: #26384a; border-
   color: #d3dbe3; }
.top-nav button:hover { background: #dce5ed; }
.top-nav select { background: white; margin-right: 8px; }
```

```
hr { margin: 14px 0 22px; border: 0; border-top: 1px solid #e0e4e8; }
h3 { margin: 26px 0 10px; padding-bottom: 7px; color: #26384a; border-bottom: 2px solid #e1e6eb; font-size: 18px; }
h4 { margin: 20px 0 9px; color: #34495e; font-size: 14px; text-transform: uppercase; letter-spacing: 0.5px; }
p { line-height: 1.5; }
code { padding: 2px 5px; border-radius: 4px; background: #eef1f4; color: #384958; font-family: monospace; }

fieldset { margin: 0 0 14px; padding: 16px; border: 1px solid #d7dce2; border-radius: 8px; background: #fafbfc; }
legend { padding: 0 7px; color: #34495e; font-weight: 600; }
fieldset label { display: inline-block; margin: 5px 0; }

.data-table { border-collapse: collapse; width: 100%; margin: 10px 0; background: white; font-size: 14px; overflow: hidden; }
.data-table th, .data-table td { padding: 9px 11px; border: 1px solid #d7dce2; text-align: left; }
.data-table th { background: #edf1f5; color: #26384a; font-weight: 600; }
.data-table tbody tr:nth-child(even) { background: #f8f9fb; }
.data-table tbody tr:hover { background: #eef4f8; }

.msg-error { margin: 12px 0; padding: 10px 12px; border-left: 4px solid #c94b4b; border-radius: 4px; background: #fff1f1; color: #a73535; }
.msg-success { margin: 12px 0; padding: 10px 12px; border-left: 4px solid #4b9561; border-radius: 4px; background: #eff9f1; color: #367247; }
.msg-empty { padding: 12px; border-radius: 6px; background: #f3f4f5; color: #666; font-style: italic; }

@media (max-width: 800px) {
    body { padding: 10px; }
    .layout-container { grid-template-columns: 1fr; grid-template-rows: auto; }
    .sidebar-left, .sidebar-right { border: none; border-bottom: 1px solid #d7dce2; }
    .top-nav form, .top-nav select, .top-nav button { width:
```

```
   100%; margin: 3px 0; }
698   .data-table { display: block; overflow-x: auto; white-space:
   nowrap; }
699 }
700 </style>
701 </head>
702 <body>
703
704 <div class="layout-container">
705 <div class="top-nav">
706 <form method="post">
707 <?= tableSelect($pdo, 'table', post('table')) ?>
708 <?= button('show_execute', 'Show') ?>
709 </form>
710 <?= formButton('show_all', 'Show All') ?>
711 <?= formButton('select', 'Select') ?>
712 <?= formButton('join', 'Inner Join') ?>
713 </div>
714
715 <div class="sidebar sidebar-left">
716 <b>DDL</b>
717 <?= formButton('create', 'CREATE') ?><br><br>
718 <?= formButton('alter', 'ALTER') ?><br><br>
719 <?= formButton('drop', 'DROP') ?><br><br>
720 <?= formButton('truncate', 'TRUNCATE') ?>
721 </div>
722
723 <div class="workspace">
724 <b>TABLE / REQUEST</b>
725 <hr>
726
727 <?php if (!empty($error)): ?>
728 <p class="msg-error"><?= h($error) ?></p>
729 <?php endif; ?>
730
731 <?php if (!empty($message)): ?>
732 <p class="msg-success"><?= h($message) ?></p>
733 <?php endif; ?>
734
735 <?php
736 if (in_array($action, ['show_execute', 'select_execute', '
```

```
   join_execute'])) {
    sendTable($rows);
}
elseif ($action === 'show_all') {
    foreach ($allRows as $tableName => $tableData) {
        echo '<h3>' . h($tableName) . '</h3>';
        sendTable($tableData);
    }
}
else {
    switch ($action) {
        case 'create':         createWorkspace(); break;
        case 'create_columns': createColumnsWorkspace(); break;
        case 'alter':          alterWorkspace($pdo); break;
        case 'drop':           simpleTableWorkspace($pdo, 'drop',
    'DROP'); break;
        case 'truncate':       simpleTableWorkspace($pdo, '
   truncate', 'TRUNCATE'); break;
        case 'insert':         insertWorkspace($pdo); break;
        case 'insert_rows':    insertRowsWorkspace($pdo); break;
        case 'update':
        case 'update_form':    updateWorkspace($pdo); break;
        case 'delete':
        case 'delete_form':    deleteWorkspace($pdo); break;
        case 'select':
        case 'select_form':    selectWorkspace($pdo); break;
        case 'join':
        case 'join_form':      joinWorkspace($pdo); break;
    }
}
?>
</div>

<div class="sidebar sidebar-right">
<b>DML</b>
<?= formButton('insert', 'INSERT') ?><br><br>
<?= formButton('update', 'UPDATE') ?><br><br>
<?= formButton('delete', 'DELETE') ?>
</div>
</div>

```

```
775 </body>
776 </html>
```

Листинг 12 — Полный код файла db_2.php